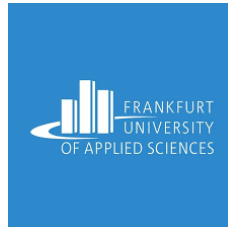


FRA-UAS
DEPARTMENT OF COMPUTER SCIENCE



REAL-TIME TRAFFIC SIMULATION WITH JAVA

SiMpFi

Instructor: Prof. Dr.-Eng. Ghadi Mahmoudi

Student 1: Kenji Bischoff - 1402720

Student 2: Nguyen Ha Khai - 1650329

Student 3: Nguyen Duy Khanh - 1648306

Student 4: To Huynh Phuc - 1651780

Student 5: Ninh Hoang Quan - 1650910



Abstract

This report provides progress, purpose, and other detailed informations about simpfi, a real-time traffic simulation app in which users can simulate real-world transportation scenarios and export useful statistics.



List of Tables

1	Team Roles	6
---	----------------------	---

List of Figures

1	Architecture Diagram	7
2	Wrapper Class Diagram	9
3	User Interface	11
4	Navigating UI	13
5	Statistics Panel	14
6	Inject Panel Interface	15
7	Map View Panel Interface	16
8	Choosing another connection in the same Intersection	17
9	Information DiaLog in Program Lights Panel	17
10	Map when entering the mouse into E45	19
11	Map when unticking E45	19
12	Inspect Panel	21
13	Filtered Export on Inspect Panel	22
14	Statistics Panel	24
15	Inject Panel	25



Contents

1	Introduction	5
1.1	Context	5
1.2	Main Issue	5
1.3	Purpose and Goals	5
2	Project Overview	5
2.1	Description	5
2.2	Technology Stack	6
2.3	Team Roles	6
2.4	Time Plan	6
3	Architecture Diagram	7
3.1	GUI & Visualization Layer	7
3.2	SUMO Integration (Wrapper) Layer	7
3.3	Data Handling (HashMap)	8
4	Class Design	8
4.1	Overview	8
4.2	Wrapper Class Diagram	8
4.3	Class Responsibilities	10
5	GUI Mockups	10
6	User Guide	11
6.1	Introduction	11
6.1.1	Overview	11
6.1.2	Target Audience	11
6.1.3	Features	11
6.2	Setup	12
6.2.1	Software Requirements	12
6.2.2	Installation Guide	12
6.2.3	Project Setup	12
6.3	User Handbook	13
6.3.1	Statistics	13
6.3.2	Inject	15
6.3.3	Map View	16
6.3.4	Program Lights	16



6.3.5	Filter	18
6.3.6	Inspect and Export	20
6.4	Technical Description	22
6.4.1	Threading and Concurrency	22
6.4.2	Error Handling and Exceptions	22
6.4.3	Performance Improvements	23
6.4.4	Technical Description - UI design	23
6.4.5	Technical Description – Statistics Panel	23
6.4.6	Technical Description – Inject Panel	24
6.4.7	Technical Description – Map View Panel	26
6.4.8	Technical Description – Program Lights Panel	26
6.4.9	Technical Description – Filter Panel	26
6.4.10	Technical Description – Inspect and Export Panel	27
7	Conclusion	28

1 Introduction

1.1 Context

SUMO - Simulation of Urban MObility - is a traffic simulation tool used to simulate urban transportation scenarios. The application is helpful for designers, researchers who design road network and evaluate features such as CO₂ levels, traffic congestion rate, etc.

1.2 Main Issue

Although providing useful functionalities for traffic simulation, SUMO is difficult to utilize effectively. In other words, only users with programming knowledge can benefit greatly from the software.

This disadvantage can be observed by the fact that users are required to grasp a basic understanding of SUMO's configurations such as XML files and how to run command lines.

1.3 Purpose and Goals

In order to address the problem mentioned above, the team aims to create a software that leverages SUMO's features but maintains the simplicity of user interface. That is, people can perform simple operations such as drag-and-drop and clicking buttons to retrieve meaningful detailed information and represent complicated traffic networks. Consequently, more and more designs can be sketched and created quickly, which potentially speeds up one's studies or transportation planning. Furthermore, an increased number of designs means that the urban area may have better transportation plans, thereby reducing traffic congestion and CO₂ emissions. Regarding study goals, the team plans to master in Object-Oriented Programming concepts and gain more experience with teamwork, writing documents such as javadoc and academic reports.

2 Project Overview

2.1 Description

The software **simpfi**, which stands for **S**imulation **Map** **T**raffic, is a traffic simulation tool used to create and monitor simulated representation of real-world networks. For clarity, the application's core is based on SUMO, but the difference is that it is developed to suit a wider range of target users from non-coders to experts.

2.2 Technology Stack

There are three main components the team uses for the project: **Java**, **TraaS**, and **SUMO**. Additionally, the team uses **GitHub** to create a shared repository for members to submit codes, and **Notion** to distribute tasks.

2.3 Team Roles

The below table shows the main role of each member during the final milestone, note that although these are specific-domain roles, the team tends to overlap the work and responsibilities in order to fully experience how to work in real-world projects:

ID	Name	Main Role
1402720	Kenji Bischoff	Inspect Panel and File Exporting
1650329	Nguyen Ha Khai	Filter Panel, Copy-On-Write for Multithread Handling, Logging & Exception Handling, and Drawing Method Enhancement
1648306	Nguyen Duy Khanh	User Interaction & Controls, Map Visualization & Rendering, Traffic Light Management, and Traffic Simulation Modes
1651780	To Huynh Phuc	UI Implementation, Multithread and Logging Handling, Testing Method Implementation, and OOP-based Architecture Redesign
1650910	Ninh Hoang Quan	Statistics Implementation, Static vs Adaptive Feedback, Vehicle Management, and Thread Separation

Table 1: Team Roles

2.4 Time Plan

For the detailed time plan, readers can see our activities in this Notion page: https://www.notion.so/2a62b60367bb80a58886c6713e08ba54?v=2a62b60367bb80db8153000caff77c02&source=copy_link.

For the GitHub repository, refer to this.

3 Architecture Diagram

Here is the architecture diagram of the project:

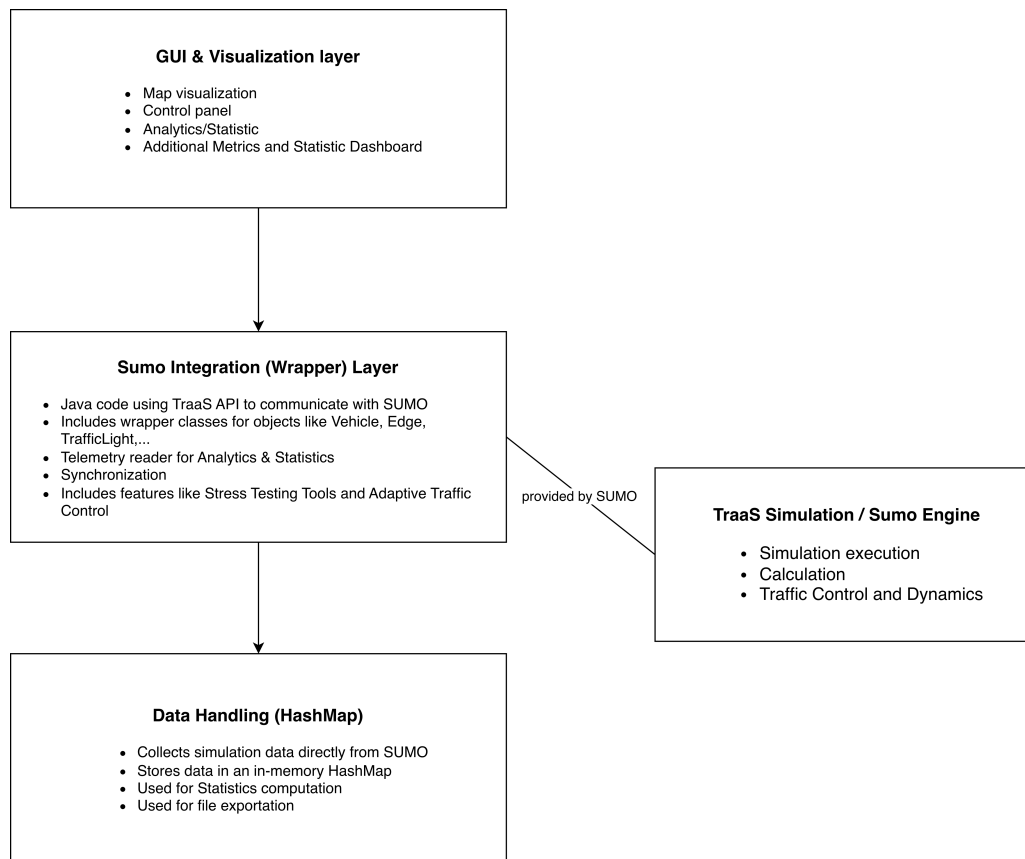


Figure 1: Architecture Diagram

From the diagram, three main layers can be observed: GUI & Visualization, SUMO Integration, and Data. Each layer provides separate methods in order to guarantee modularity and coherence in the system's design.

3.1 GUI & Visualization Layer

The main task of this layer is to present a user-friendly UI for users. In particular, it includes map visualization, multiple panels such as **Inject**, so that users can add vehicles to the network. Additionally, users can move and scale the map by dragging and scrolling the mouse, respectively.

3.2 SUMO Integration (Wrapper) Layer

The layer is where TraCI connection takes place. Put differently, it is the interface where SUMO and wrapper classes communicate with each other. From there, the simulation can be

executed, logical operations like numerical calculations can be performed, and network controls can be handled flawlessly.

3.3 Data Handling (HashMap)

As its name suggests, this layer collects simulation data from the SUMO server and stores it in a HashMap, which is essential for exporting meaningful statistics in PDF or CSV files. Also, the layer is developed to ensure the consistency and integrity of data throughout different parts of the system.

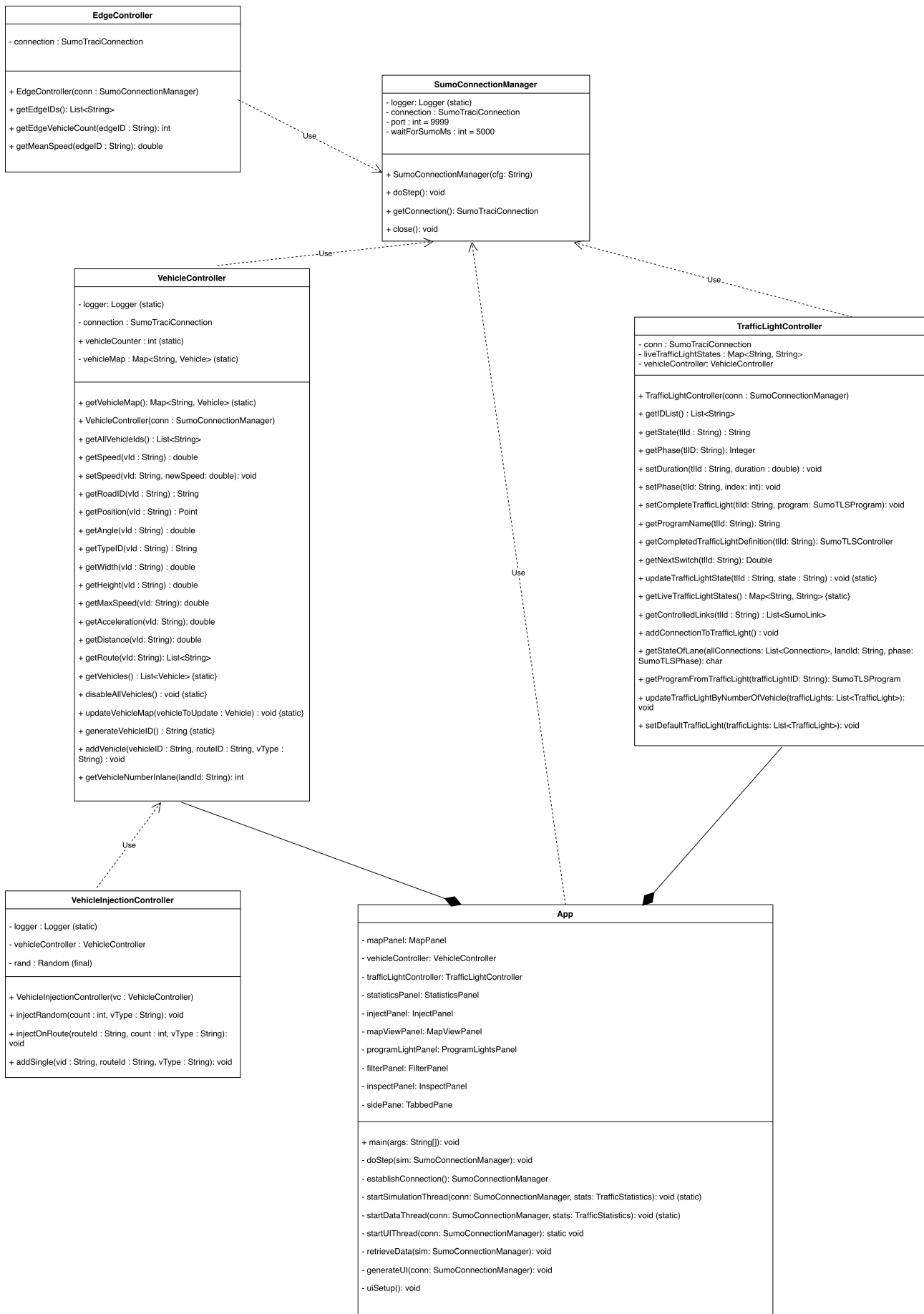
4 Class Design

4.1 Overview

Besides core wrapper classes, the team decided to draw the user interface by defining and connecting classes such as buttons, panels,... As a result, the number of classes was up to 30, which would be excessively complicated and difficult to understand if the team sketch them in one single class diagram. Therefore, a wrapper class diagram was designed to ensure the conciseness and readability.

4.2 Wrapper Class Diagram

The class diagram is shown as follows:



4.3 Class Responsibilities

- **EdgeController**: gets the information of edges and vehicles to calculate meaningful statistics such as average speed on a specific edge.
- **VehicleInjectionController**: controls different types of injections like batch, batch random or single mode.
- **VehicleController**: its main role is to process and retrieve detailed information regarding vehicles. To achieve this, the class has a `SumoTraciConnection` attribute `connection` used to actively connect to SUMO so that getters such as `getSpeed()`, `getRoadID()` can be implemented. Additionally, `vehicleCounter` and `vehicleMap` are developed to keep tracks of vehicles number and mapping. Furthermore, the class provides several useful methods like `addVehicle()` for users to perform vehicle-related operations.
- **TrafficLightController**: as its name suggests, the class deals with traffic light objects, especially in the real-time scenarios since it has a live traffic light mapping: `liveTrafficLightStates`. Similar to `VehicleController`, the class establishes a connection with SUMO using the attribute `connection` and therefore is able to implement getters such as `getIDList()`, `getState()`. Besides, `updateTrafficLightState()`, `getLiveTrafficLightStates()` are provided to maintain the logic of SUMO traffic lights in the Simpfi's map.
- **SumoConnectionManager**: is the class that creates and manages the TraCI connection between SUMO and the project's app. This is achieved by passing a XML configuration file to its constructor and then the program can initialize the connection by `getConnection()` method and close it safely later using `close()`. Moreover, it provides an *important* method `doStep()` which advances the simulation by one timestep.
- **App**: is where the main function resides. Particularly, it connects the frontend (`mapPanel`) and the backend (`vehicleController`, `trafficLightController`, ...); defines the TraCI connection; and handles the connection loop so that the software can function properly. Moreover, it is where multi-threading takes place.

5 GUI Mockups

Below is the intended final user interface of the project:

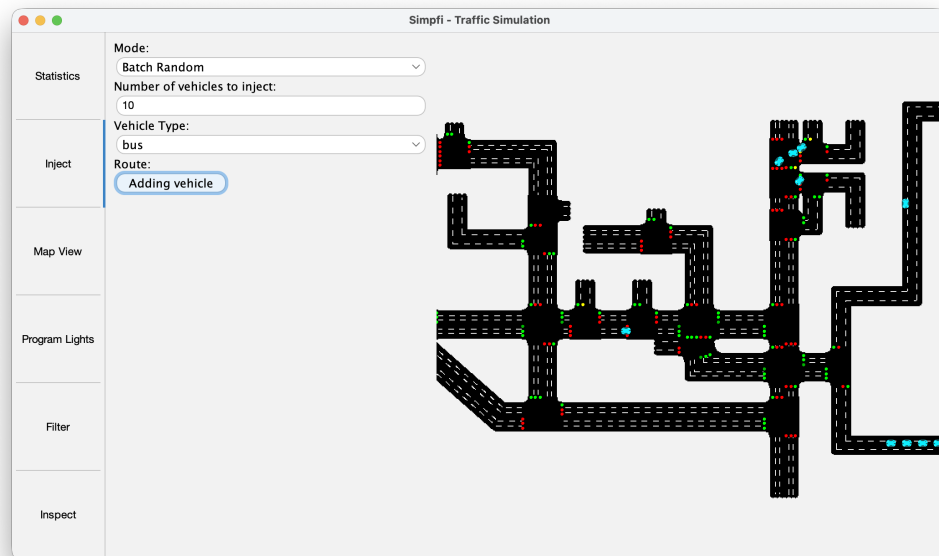


Figure 3: User Interface

6 User Guide

6.1 Introduction

6.1.1 Overview

This application provides a simple graphical interface for controlling, visualizing, and analyzing SUMO traffic simulations. Users can inject traffic, program traffic lights, and monitor simulation statistics in real time.

6.1.2 Target Audience

Our project is aimed at users working with traffic and urban mobility simulations, such as:

- Designers.
- Transportation Researchers.
- Environmental Analysts.
- Students.

6.1.3 Features

- Real Time Map Visualization.

- UI including:
 - Vehicle Injection
 - Vehicle Inspection and Filtering
 - Traffic Light Programming
 - Statistics Panel
 - Map View settings

6.2 Setup

6.2.1 Software Requirements

1. Java 17 (JDK) or higher.
2. SUMO software.
3. IDE for Java.

6.2.2 Installation Guide

Make sure that you download the versions compatible with your operating system.

1. Install Java Development Kit (JDK from oracle).
2. Install SUMO (Simulation of Urban MObility).
3. Install an IDE of your choice to run the Java project (Eclipse, VSCode, IntelliJ, ...).
4. Install `flatlaf-3.6.2.jar`, `jcommon-1.0.23`, `jfreechart-1.0.19`, TraaS.

6.2.3 Project Setup

The last step of the setup is to download our project by cloning it from GitHub. After that, open it up in your IDE.

For running the application, simply run the class "App.java".

6.3 User Handbook

The application window will be divided into 2 main sections: (1) Map Panel on the right side and (2) Control Panel on the left side.

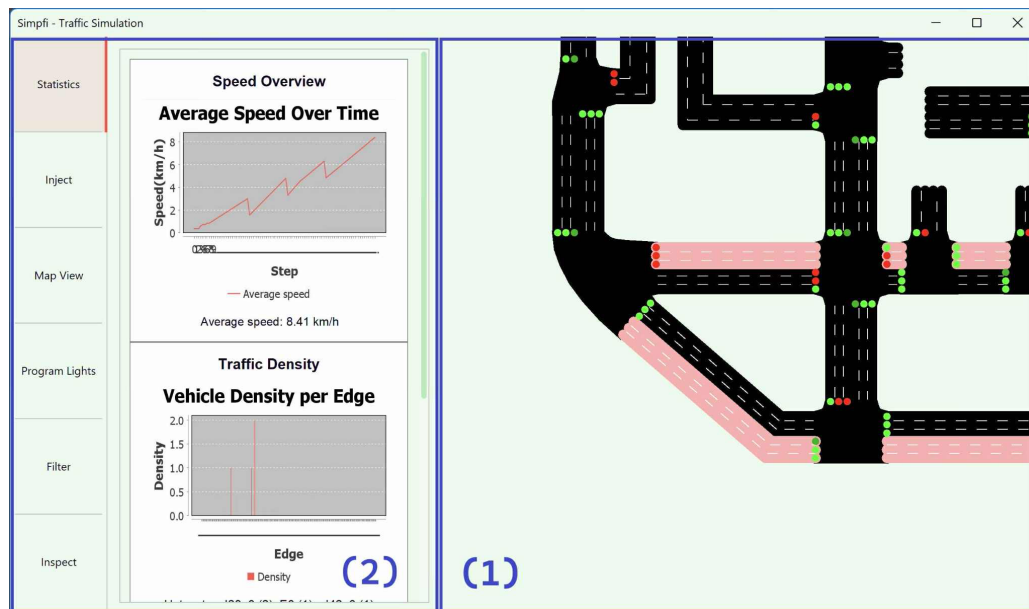


Figure 4: Navigating UI

The **Map Panel** can be manipulated in 2 ways:

- Move around with arrow keys, zoom in with “Ctrl =” and zoom out with “Ctrl -” (Not recommended since arrow keys can also be used to move around in the Control Panel and differences between US keyboard layout and German keyboard layout makes zooming more difficult).
- Move around by clicking and dragging the map area, zoom in by scrolling up and zoom out by scrolling down (Recommended).

The **Control Panel** have six sub-panels with different features:

6.3.1 Statistics

Shows a variety of charts that is updated in real time with statistics from the running simulation:

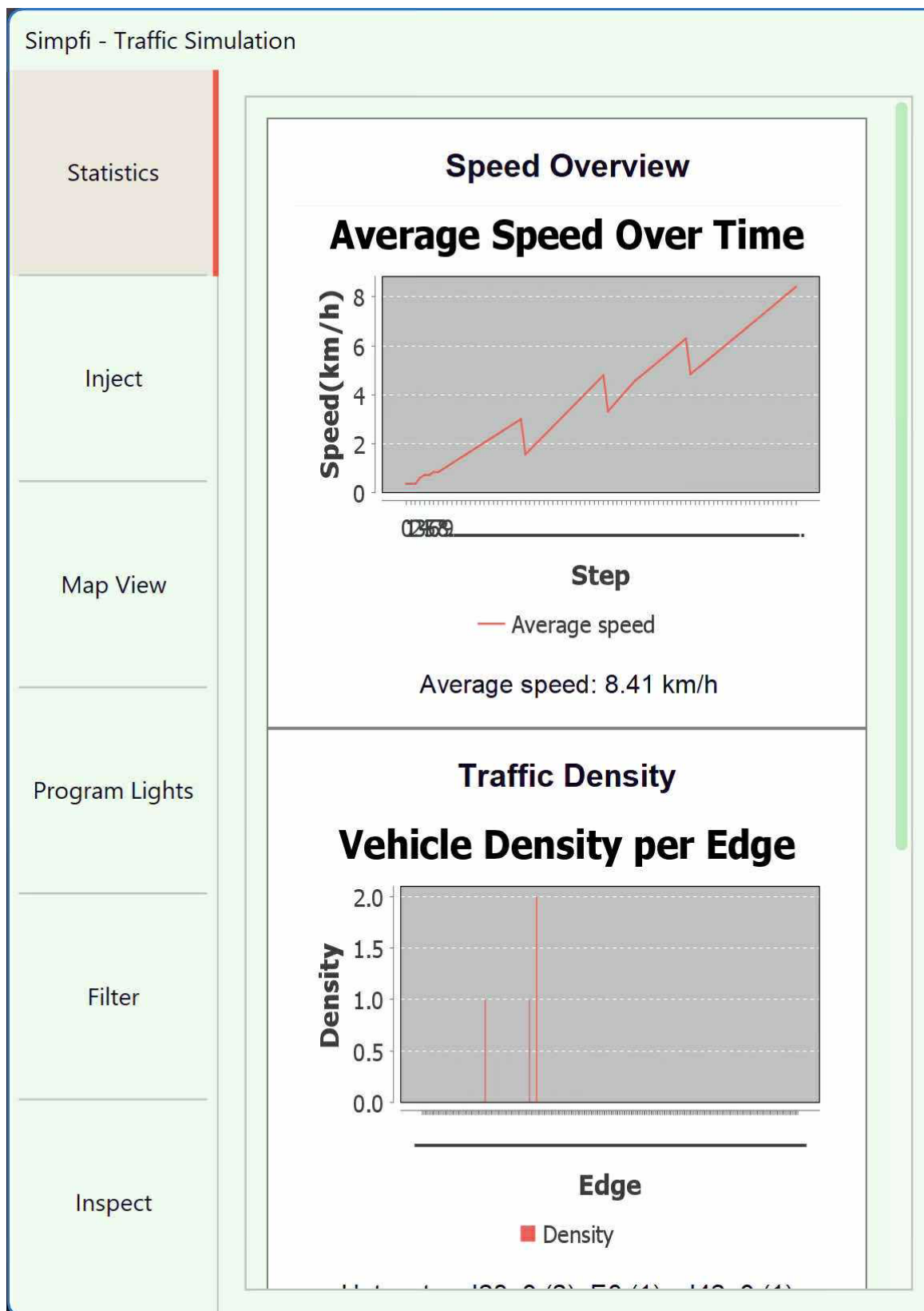


Figure 5: Statistics Panel

6.3.2 Inject

In the Inject Panel, there are four Dropdowns and one button to add vehicle to the network:

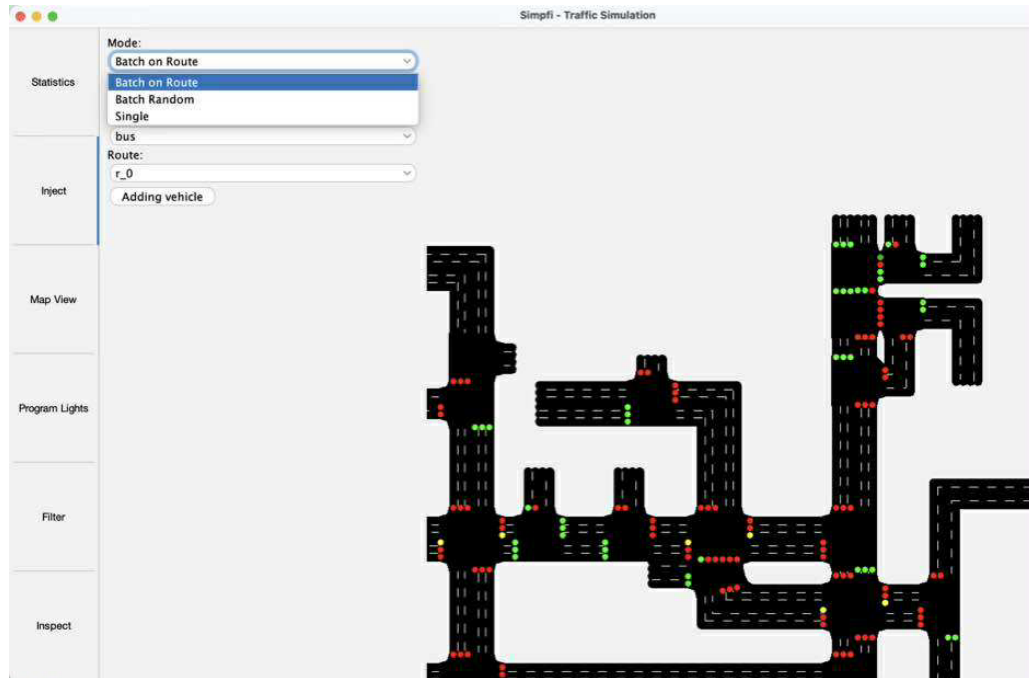


Figure 6: Inject Panel Interface

Before injection, users have to choose one of three modes: batch on route, batch random, and single. Batch On Route means that users can inject any number of vehicles they want on a specific route chosen from the Route Dropdown. For the Batch Random mode, users are allowed to add any number of vehicles to random routes. The last mode is Single mode, for which users can only inject one vehicle on one route.

After choosing the mode, users have to set a value indicating the number of vehicles that they want to inject. This Dropdown only appears when a chosen mode is Batch On Route or Batch Random. The third Dropdown is Vehicle Type, where users can choose one of five vehicle types. Each vehicle is colored differently depending on its type. Next, users have to specify which route they want their vehicles to run. This Dropdown appears only when a mode is Batch On Route or Single. Finally, users press the button "Adding vehicle" to start the process.

6.3.3 Map View

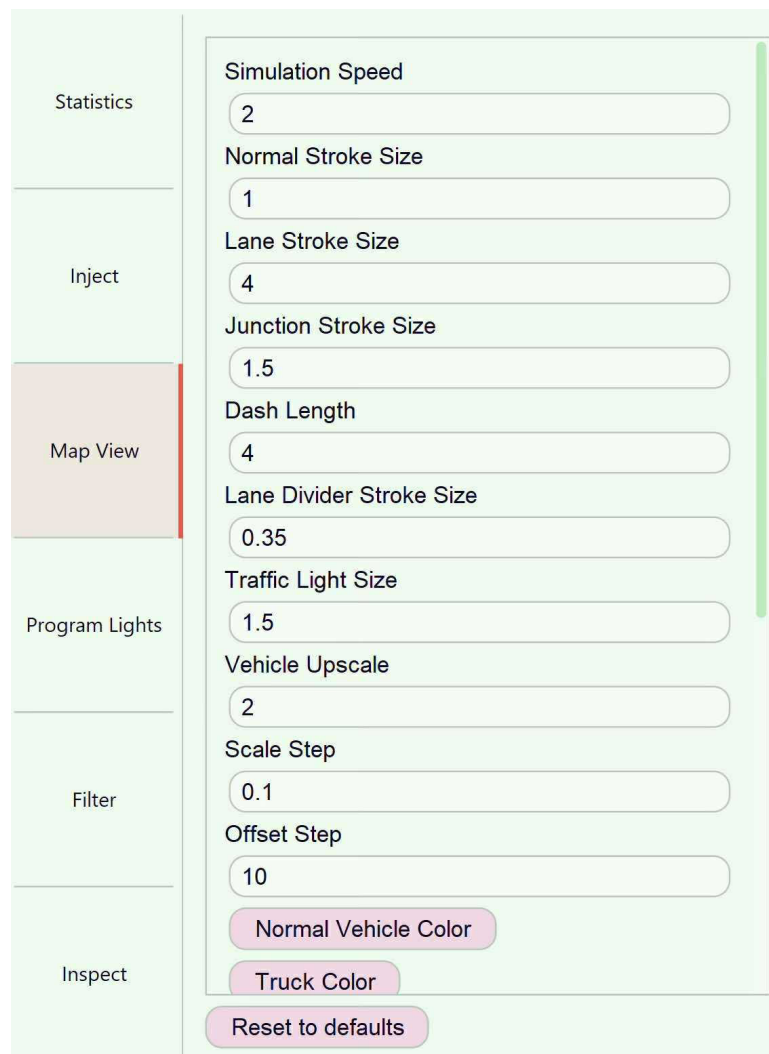


Figure 7: Map View Panel Interface

In the Map View panel, user can change many settings about the display of the map such as how big the elements are or what color is used. **Note** that set the “Simulation Speed” too high is not intended and can have unexpected consequences.

6.3.4 Program Lights

In this panel, users can interact with traffic lights in the network. To start with, users need to identify the traffic lights they want by selecting the traffic light ID in “Select intersection”.

After a intersection is chosen, one of its connections is highlighted. The intersection between two lanes connection is the junction that users choose. If users want to see other connections, simply select them in “Connection”. For example, another connection becomes highlighted after being selected:

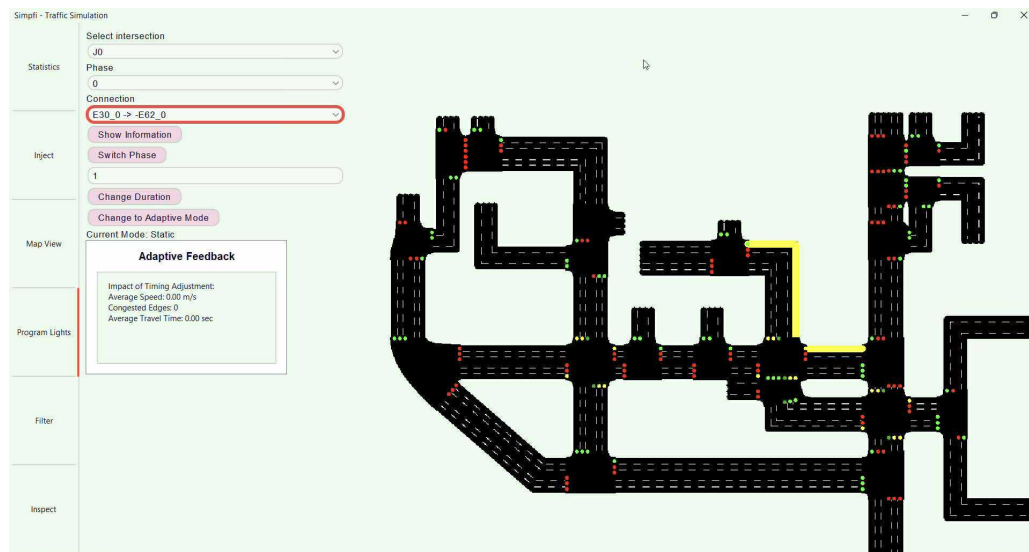


Figure 8: Choosing another connection in the same Intersection

If users want to view all the information including the state of the traffic light, the current phase, the signal of each connection, and the remaining time of the current duration, Users can click on “Show information”.

After that, a pop-up dialog will appear on the screen and update by real time in traffic light. And the signal of traffic light will represent the status light of that connection. Here “r” is red, “y” is yellow, and “g” is green. Sometimes, users will see big “G”, which represents for green light, but higher priority than “g”.

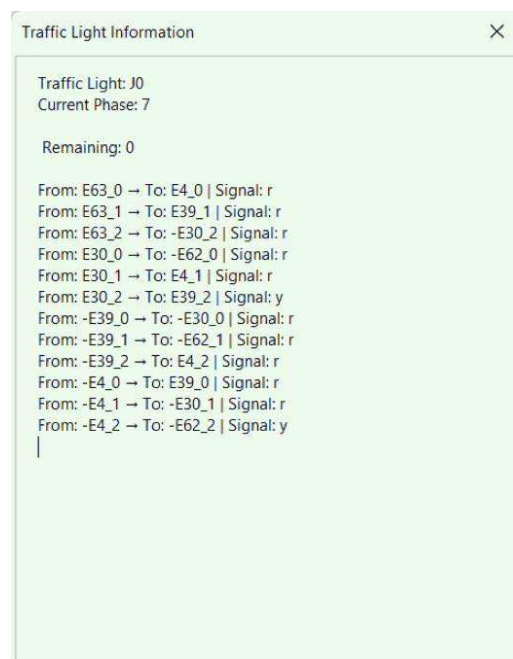


Figure 9: Information DiaLog in Program Lights Panel

If users want to switch the desired phases immediately. First, users need to choose the traffic light ID in “Select intersection” and then choose the phase number in “Phase” and click “Switch Phase” after that to trigger the event. Similarly, if users want to set new duration for specific phases of the traffic light. First, select traffic light ID in “Select intersection”. And then select the phase user want to choose in “Phase”. Then enter the time value which is the new duration for the phase user choose. And finally, click “Change Duration”. Additionally, this panel introduce two different traffic light modes: static and adaptive mode. Users can freely switch between two modes by clicking “Change to Adaptive Mode” or “Change to Static Mode”. Here “Static Mode” is when the logic or program of the traffic light is predefined from the *XML* file. So, there is no change in the definition of the program (e.g. phases, states, signals, duration, etc.). If users tend to choose the mode to “Adaptive” mode. The traffic light may change based on the congestion of the traffic. Here, the team added the logic that if any traffic light has the lane which the number of vehicles of that lane is larger than 2, then the duration of current phase of that traffic light is increased to 5 (In other words, take the default duration and plus 5). This helps users to track the traffic and have a comparison between static mode and adaptive mode.

6.3.5 Filter

This panel provides three main features:

- **vehicle-type filter:** users can see a list of checkboxes showing different types of vehicles. Vehicles whose type is not ticked will not be painted on the map.
- **road filter:** with this filter, users are able to filter out roads, which are the combinations of edges and their sub-edges(not include junction edges). Designed similarly like the **vehicle-type filter**, users check the boxes to opt in roads where vehicles are visible. The only difference is that users can enter the mouse into each checkbox to see the corresponding road. For example, E45 is entered:

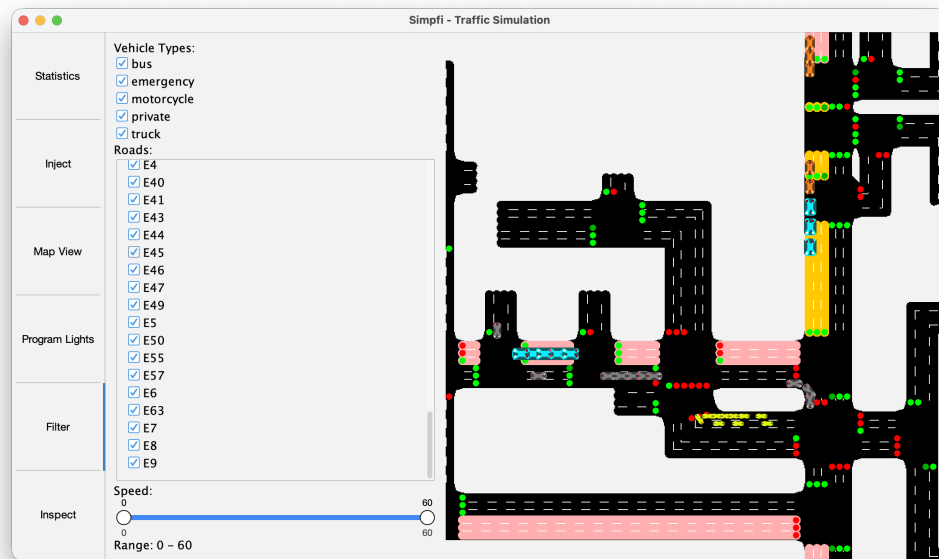


Figure 10: Map when entering the mouse into E45

Then when E45 is unticked, vehicles become invisible in that road:

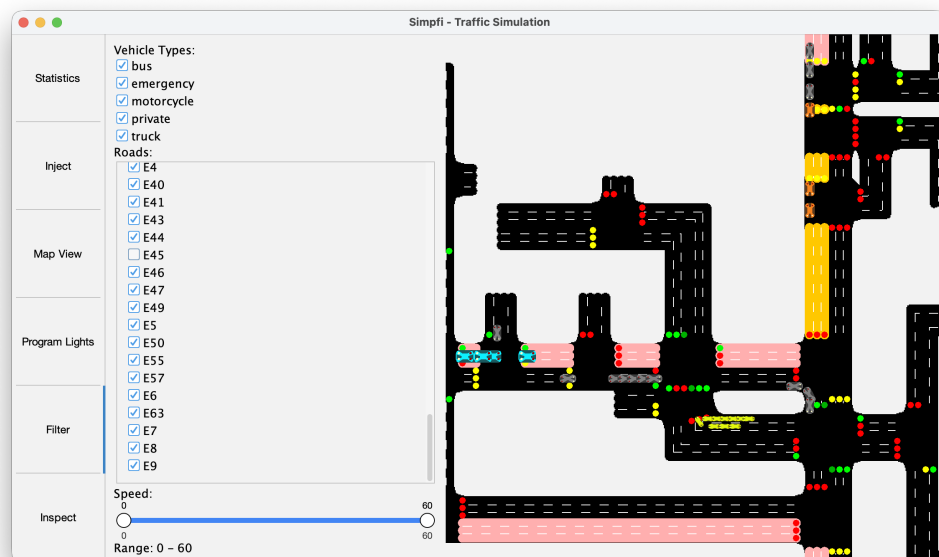


Figure 11: Map when unticking E45

- **speed filter:** Different from the two previous filters, this filter use a range slider to filter out vehicles within a specific range. The defined range is set from 0 to 60(m/s), which is derived from German speed limits. Users can drag two thumbs to set the desired speed range for visible vehicles.

6.3.6 Inspect and Export

With the **Inspect and Export Panel**, users can select vehicles directly from the Simulation by clicking on it. Users can also export simulation statistics to CSV or generate a PDF containing metrics and a congestion chart.

The user can switch between two modes by pressing the **Change Mode** Button on the bottom: The **Pan mode** is used to drag the map around. After changing to the **Select mode**, the user will be able to select a vehicle on the map by clicking on it.

The list on the top left of the panel shows the currently selected Vehicles. To the right there is a **Select All button** to select all vehicles, which are currently in the Simulation. Use the **Clear button** below to clear the List.

If the user chooses a selected vehicle from the upper box, this element will be highlighted and the corresponding stats will show in the lower Box.

The list can also be sorted by using the Group By dropdown. The user can group the selected vehicles by **Vehicle Type**, **Color**, **Speed** or **Route**. To see the list without grouping, select **None**.

The stat **Vehicle Type** shows one of the following vehicles: Bus (blue), Emergency (orange), Motorcycle (pink), Private (yellow), Truck (grey).

The stat **Color** shows the color in an RGB format, representing the Color components Red, Green, Blue from 0 to 255. Example: R:255, G:0, B:0 $\rightarrow$ maximum (255) amount of Red, 0 Green and Blue

The **Speed** is the only dynamic statistic in the list. It gets updated every 500ms. The speed is given in km/h.

Max Speed shows the maximum speed the vehicle reaches in the simulation.

Acceleration is given in m/s^2 .

The **Distance Traveled** stat shows the covered distance in this simulation and is given in meters.

The **Route** stat shows the connected edges forming the vehicles route.

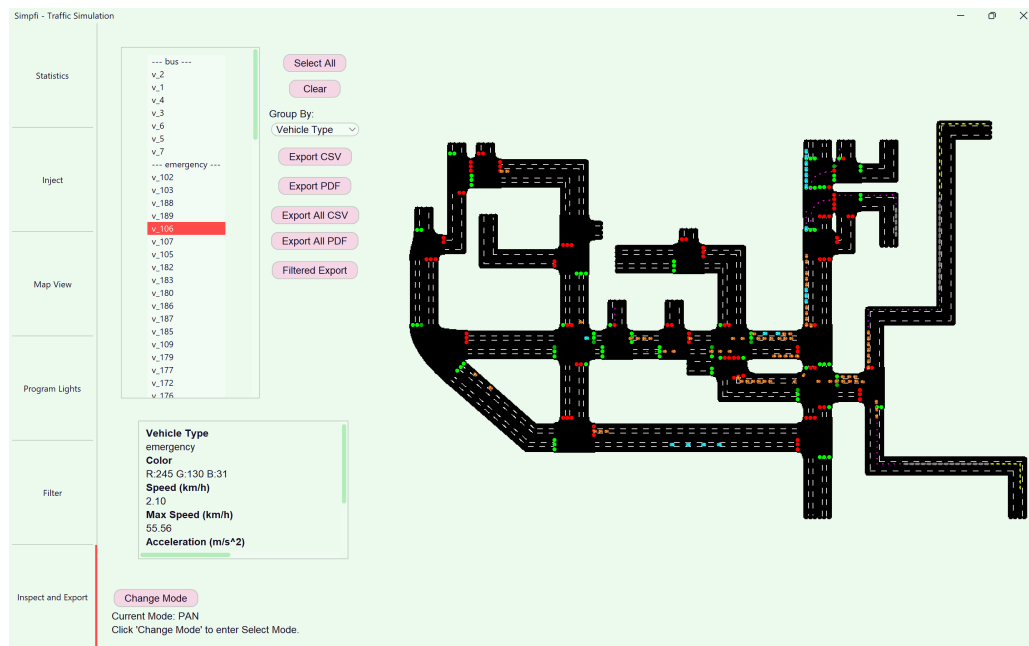


Figure 12: Inspect Panel

As you can see in the picture, the list is currently grouped by Vehicle Type. Each batch of vehicles with the same type are headed with their types name.

There are various options for exporting simulation data. The first option enables users to select between exporting all vehicles either to CSV or as PDF by clicking on **Export All CSV** or **Export All PDF**. When wishing to export only the selected vehicles showing in the list, the user can either use the **Export CSV** or **Export PDF** button. For the export of specific vehicles, the user can use the **Filtered Export** button. It opens up a new window, in which the user can filter the export data by using the tickboxes and the traveled distance range feature. The user can hover over the filters for more information. When finished with the filtering, the user can export the data via CSV or as PDF. When exporting, the File Chooser pops up to enable the user to save their data at the desired spot. The CSV file contains the Vehicle ID, vehicle type, speed, maximum speed, acceleration, distance and route values for each vehicle. The PDF contains the same information, just without the route data. In addition it contains a chart, which shows what edges in the simulation are **congested**. There need to be **five or more vehicles on an edge** for it to count as **congested**. Without this condition it will just show that the vehicles are not available (N/A). The chart shows which edges have how many vehicles on it at the time of the export. The different colors of the graph indicate how heavily the edge is congested (green = low, yellow = medium, red = high). Here you can see how the filtered export dialog looks like:

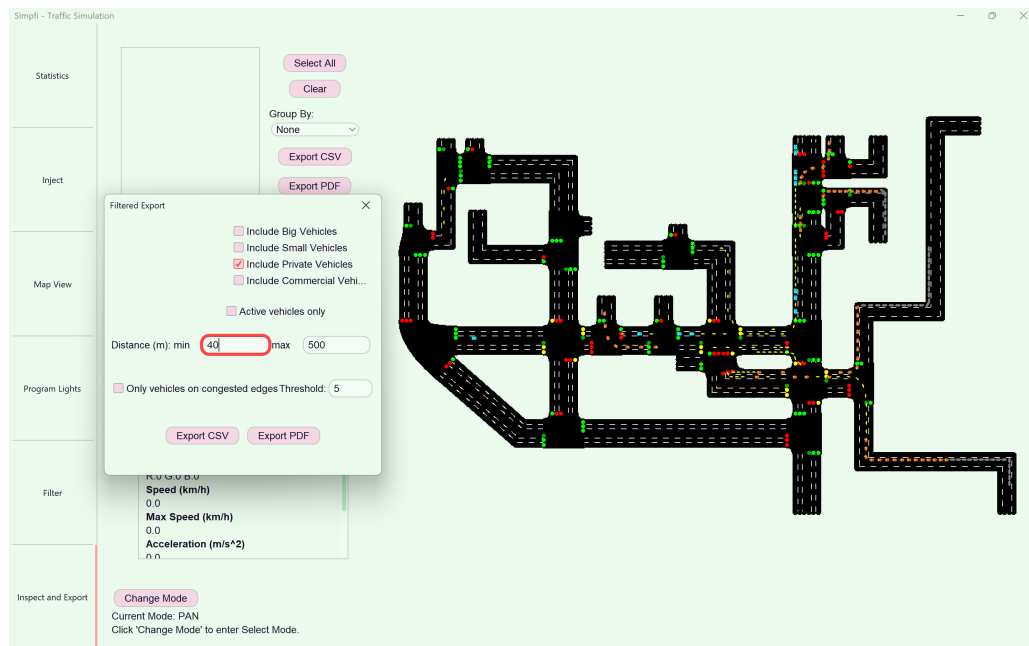


Figure 13: Filtered Export on Inspect Panel

6.4 Technical Description

6.4.1 Threading and Concurrency

The project used three main threads:

- **UI Thread:** Its main job is to update the visual display (`MapPanel`). Besides that, it uses `SwingUtilities.invokeLater()` to ensure all updates happen on the Event Dispatch Thread (EDT) for thread safety.
- **Simulation Thread:** Advances the SUMO simulation, runs the main simulation loop at a controlled timestep. Moreover, it uses a lock to ensure thread-safe access to shared resources.
- **Data Thread:** Monitors when the simulation steps forward, updates traffic statistics after each simulation step, ... Most importantly, it updates vehicles and traffic lights data for controllers and snapshot.

6.4.2 Error Handling and Exceptions

Proper logging was implemented thoroughly via `java.util.logging.Logger`. In the source code, readers can also observe a `logging.properties` file used to set up the suitable logging level for each package and define writing approaches (write to files and consoles). In particular, messages can be viewed in `app.log` located in the `logs` folder.

In addition to error handling, the team created a customized exception `InvalidNetworkConfigurationException`. It is thrown when the returned network does not meet essential requirements (e.g. missing edges). By developing own exception, the team benefited from its semantic meaning and there it aided to locate the errors more efficiently.

6.4.3 Performance Improvements

Previously, the entire map is repainted after a certain period of time, which can be resource-wasting if some components never move. In order to improve that, the team defined static layer including edges and junctions to avoid repainting it unless specific interactions from users happen like zooming the map. Also, vehicles outside the map will also not be drawn (lazy drawing), which lessen the work effectively.

Additionally, copy-on-write are implemented to draw the user interface without having another lock in `MapPanel.java`. Furthermore, the project is redesigned to leverage more OOP concepts such as interface (`Drawable.java`). Also, the team restructured files in order to enhance the cleanliness of code.

6.4.4 Technical Description - UI design

In order to render the main map along with other core features, the team leveraged basic components (`JButton`, `JPanel`,...) which extends Java Swing classes and overrode `paintComponent()` method. This approach gave the team a great opportunity to customize and design the visuals of traffic components such as edges, traffic lights, vehicles,...

Nevertheless, since there are only a few selections of built-in UI components to choose from, more efforts must be made when the team attempted to include advanced features in the application. For example, `RangeSlider`, which is available in `JavaFX`, must be implemented manually by one of the team members to meet the functional requirements.

6.4.5 Technical Description – Statistics Panel

In the Statistics feature, the average speed, vehicle density per edge, and travel time distribution are visualized by 3 live charts: line chart, bar chart, and histogram. First of all, the average speed is computed by summing all vehicles' velocities and dividing by the number of vehicles running on the network via the `getAverageSpeed()` method. The second data is the traffic density, which simulates the density of vehicles on each edge using a bar chart and results in congested hotspots – 3 edges with the largest number of vehicles. Particularly, the number of vehicles is collected on each edge per step by traversing all edge IDs and storing them in a hash map, which is the `edgeVehicleCount`. Then, this data is put into the `vehicleDataset` for plotting a live bar chart. Finally, the travel time data is computed by subtraction of the current step

from the start step. Then, it is stored in a list `travelTimes` for the travel time distribution visualization.

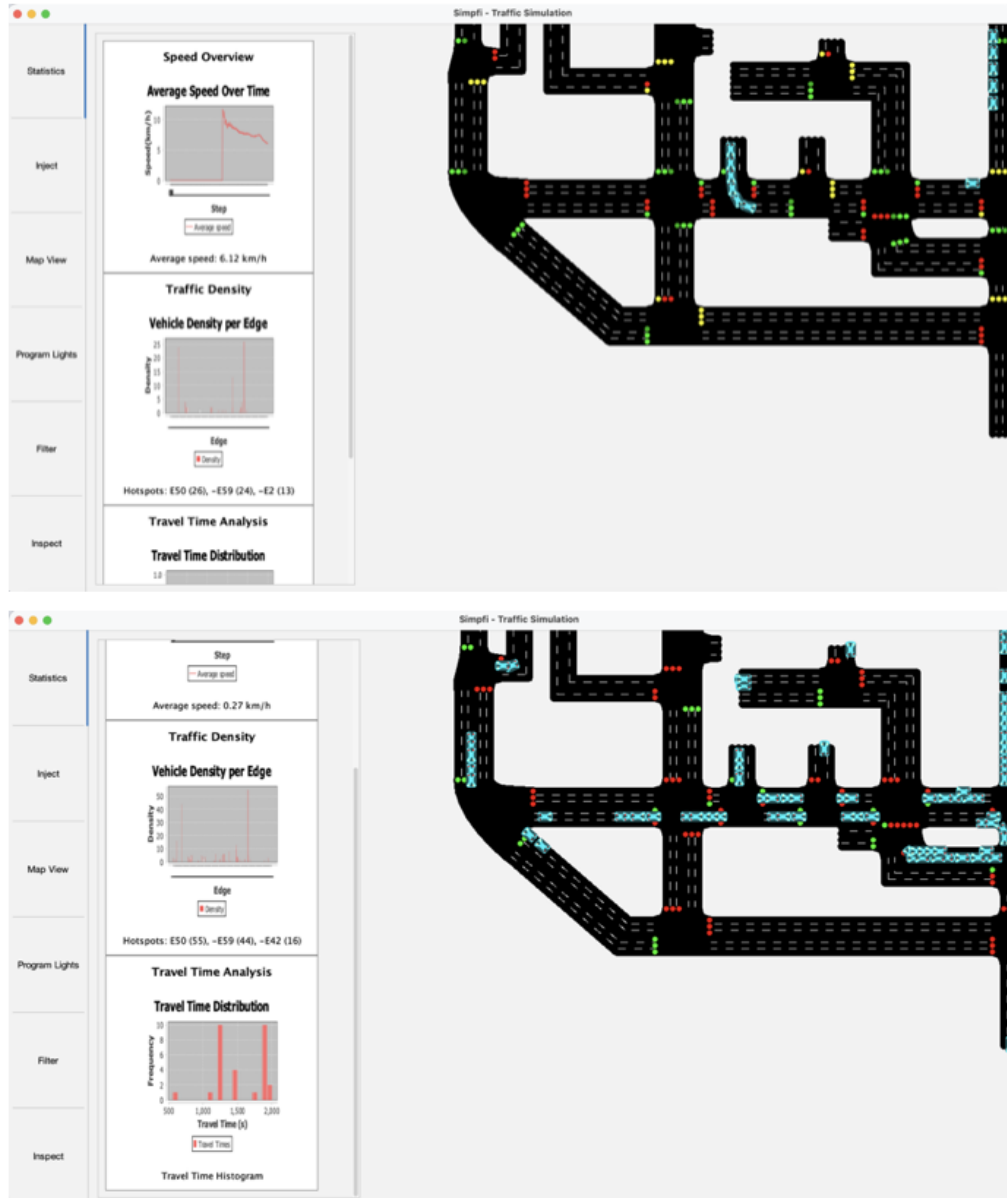


Figure 14: Statistics Panel

6.4.6 Technical Description – Inject Panel

The `InjectPanel` class is a Java Swing-based user interface for injecting vehicles. It inherits the `Panel` class and allows for selection of vehicle types, routes, injection mode, and the number of vehicles to inject. The `InjectPanel` allows users to dynamically inject vehicles into a network during simulation time. It supports three modes of injection:

- **Batch On Route** – inject multiple vehicles across a specific route

- **Batch Random** – inject multiple vehicles across a network
- **Single** – inject a selected vehicle across a chosen route

The image below shows 5 key components of the Inject Panel feature:

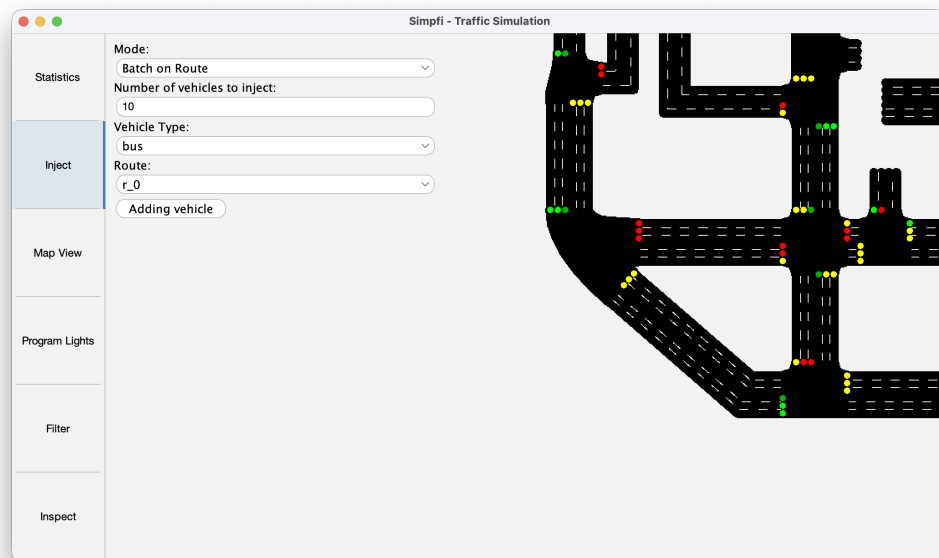


Figure 15: Inject Panel

1. **Mode Dropdown (modeDropdown)**: Provides 3 options – Batch On Route, Batch Random, and Single. It determines whether users want to inject multiple vehicles into a selected or a random route, or even a single vehicle into a specific route when clicking on the “Add vehicle” button.
2. **Count Field (countField)**: Provides a text area allowing users to type any number of vehicles they want to add to a network in batch modes. In a single mode, it is hidden dynamically.
3. **Vehicle Type Dropdown (vehicleTypeDropdown)**: allows users to select a vehicle type from a list of vehicle types obtained from `Settings.network.getVehicleTypes()`, which is wrapped by the `getAllVehicleTypesAsString()` method.
4. **Route Dropdown (routeDropdown)**: allows users to select a route for vehicle injection from a list of route IDs achieved from `Settings.network.getRoutes()`, which is wrapped by the `getAllRouteIdsAsStrings()` method.
5. **Add Vehicle button (addVehicleBtn)**: Executes the vehicle injection process via mouse click in a separate Thread to prevent blocking the EDT from Java Swing.

The `addVehicle()` method handles vehicle injection and reads users' input for vehicle mode, vehicle count, vehicle type, and route. Then the method executes a thread for injection and divides three modes into three cases using three different methods obtained from the `VehicleInjectionController` class.

6.4.7 Technical Description – Map View Panel

In the map view panel, many attributes of the map can be changed freely by the user. This is possible by updating an internal variable that can be accessed by all classes and functions so that the map drawing can be consistent. In order to implement the "Reset defaults" button, another list of constants are stored with the default values, and when the button is pressed, all properties of the map are set to the original default value.

6.4.8 Technical Description – Program Lights Panel

The Program Light Panel provides access to all traffic light information, including the junction ID, all lane connections, and the traffic light programs defined in SUMO. These attributes are retrieved through SUMO's junction calls. Since the drawing interacts with the SUMO API (TrasS), traffic light states can be changed dynamically by calling the SUMO traffic light interface. User interface elements such as drop-down menus, buttons, text areas, and dialogs are implemented by extending Java Swing classes. The remaining duration of the current phase of specific traffic light is computed by retrieving the next switch time from SUMO and subtracting it by the current time. The signal of each traffic light connection is parsed from the traffic light state String by splitting it into individual characters, where each character represents the signal assigned to a specific connection. Moreover, to ensure consistency and accuracy, both the remaining duration and the signal of the traffic light are updated in real-time and displayed in "Show Information". Finally, in order to implement the logic of "Adaptive mode", the map is continuously updated in real time. In this mode, the update logic adjusts the traffic light phase duration based on the current traffic flow conditions, enabling the system to adapt automatically and maintain flexible traffic signal control.

6.4.9 Technical Description – Filter Panel

For the implementation of the first two filters(vehicle-type and road), the team defined a `filterFlag` in `VehicleType.java` and `Road.java`. After creating the dropdowns, which extend `JComboBox`, listeners were added to those components to update the `filterFlag` when there are updates. As a result, the map will only draw vehicles whose `filterFlag` is set to `true`.

In addition to two filters above, speed filter is implemented in a more simple approach, since

each vehicle already has its own `getSpeed()` method and the code just needs to check if its speed falls within the limited range before drawing. However, the difficult part is the Range Slider, which is not supported in Java Swing. Therefore, the team developed a new `RangeSlider` object that inherits from `JComponent` to display it and let users freely choose the desired speed range. Furthermore, the maximum speed was set to 60 m/s manually because the team could not find any specific documentation about vehicles' maximum velocity in SUMO, so this figure was decided based on our research about speed limits in Germany.

6.4.10 Technical Description – Inspect and Export Panel

The Inspect and Export Panel allows users to inspect the vehicles in the simulation. Users can toggle between two modes:

- **Pan Mode:** Enables map navigation by dragging.
- **Select Mode:** Allows selecting a vehicle on the map with a click.

It also enables the user to export simulation data via CSV or as PDF. The **Select Mode** feature is realized by a Mouse Listener to get the coordinates of the selected part of the map. These coordinates are then translated to world coordinates to be utilized to find the nearest vehicle in a range of 20 units.

The top-left list displays the currently selected vehicles. Users can use the **Select All** button to select all vehicles in the simulation or the **Clear** button to clear the list. Selecting a vehicle from the list marks it and shows its statistics in the lower box. The list can be sorted using the Group By dropdown. Users can group the selected vehicles by Vehicle Type, Color, Speed, or Route. To display the list without grouping, select None. The speed headers, which appear when grouping by route are not updated live, because this would be too demanding.

Displayed vehicle statistics include:

- **Vehicle Type:** Bus, Emergency, Motorcycle, Private, Truck.
- **Color:** RGB format (0-255) representing Red, Green, and Blue components.
- **Speed:** Updated every 500 ms, in km/h.
- **Max Speed:** Maximum speed reached during the simulation.
- **Acceleration:** In m/s^2 .
- **Distance Traveled:** Covered distance in meters.
- **Route:** Sequence of connected edges forming the vehicle's route.

Simulation data can be exported by using the five buttons on the right hand side below the Group by dropdown. For exporting only the selected vehicles, there are the two buttons **Export CSV** and **Export PDF**. The buttons **Export All CSV** and **Export all PDF** export the data of all vehicles, which are currently in the simulation. The **Filtered Export** button opens up a dialog with various tickboxes and filters:

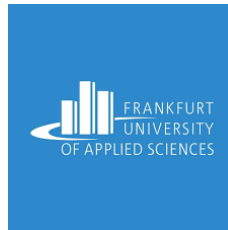
- **Include Big Vehicles:** Includes trucks, buses and emergency vehicles.
- **Include Small Vehicles:** Includes private vehicles and motorcycles.
- **Include Private Vehicles:** Include vehicles with the VehicleType private.
- **Include Commercial:** Includes trucks and buses.
- **Active Vehicles Only:** Include only vehicles, that are still active in the simulation.
- **Distance:** Select a minium and maximum traveled distance to filter vehicles by distance.
- **Congested Edges:** Only select vehicles on a "congested edge", where the number of vehicles $\geq$ Threshold (default = 5).

The data selected can then be exported via CSV or as PDF. Clicking on any of the export buttons opens up the File Chooser, where the user gets a suggested file name, with a file extension enforcement, which secures the file ending to be either **.csv** or **.pdf**. The CSV file contains the Vehicle ID, vehicle type, speed, maximum speed, acceleration, distance and route data for each vehicle. The PDF contains the Vehicle ID, vehicle type, speed, maximum speed, acceleration and distance traveled data. The PDF also contains a chart, which shows the congested edges. The chart only shows edges with five or more vehicles on it. It shows the Edge ID and how many vehicles are on it at the time of the export. The color of the graph indicates congestion (green = low, yellow = medium, red = high).

7 Conclusion

The team has successfully completed the project within a given deadline and gained invaluable knowledge and experience through this.

FRA-UAS
DEPARTMENT OF COMPUTER SCIENCE



REAL-TIME TRAFFIC SIMULATION WITH JAVA

SiMpFi

Instructor: Prof. Dr.-Eng. Ghadi Mahmoudi

Student 1: Kenji Bischoff - 1402720

Student 2: Nguyen Ha Khai - 1650329

Student 3: Nguyen Duy Khanh - 1648306

Student 4: To Huynh Phuc - 1651780

Student 5: Ninh Hoang Quan - 1650910



Abstract

This report provides progress, purpose, and other detailed informations about simpfi, a real-time traffic simulation app in which users can simulate real-world transportation scenarios and export useful statistics.



List of Tables

1	Team Roles	5
---	----------------------	---

List of Figures

1	Architecture Diagram	6
2	Wrapper Class Diagram	8
3	User Interface	10



Contents

1	Introduction	4
1.1	Context	4
1.2	Main Issue	4
1.3	Purpose and Goals	4
2	Project Overview	4
2.1	Description	4
2.2	Technology Stack	5
2.3	Team Roles	5
2.4	Time Plan	5
3	Architecture Diagram	6
3.1	GUI & Visualization Layer	6
3.2	SUMO Integration (Wrapper) Layer	7
3.3	Data Layer	7
4	Class Design	7
4.1	Overview	7
4.2	Wrapper Class Diagram	8
4.3	Class Responsibilities	8
5	GUI Mockups	10
6	User Guide	10
6.1	Introduction	10
6.1.1	Overview	10
6.1.2	Target Audience	10
6.1.3	Features	11
6.2	Setup	11
6.2.1	Software Requirements	11
6.2.2	Installation Guide	11
6.2.3	Project Setup	11
6.3	Getting Started	12
7	Conclusion	12

1 Introduction

1.1 Context

SUMO - Simulation of Urban MObility - is a traffic simulation tool used to simulate urban transportation scenarios. The application is helpful for designers, researchers who design road network and evaluate features such as CO₂ levels, traffic congestion rate, etc.

1.2 Main Issue

Although providing useful functionalities for traffic simulation, SUMO is difficult to utilize effectively. In other words, only users with programming knowledge can benefit greatly from the software.

This disadvantage can be observed by the fact that users are required to grasp a basic understanding of SUMO's configurations such as XML files and how to run command lines.

1.3 Purpose and Goals

In order to address the problem mentioned above, the team aims to create a software that leverages SUMO's features but maintains the simplicity of user interface. That is, people can perform simple operations such as drag-and-drop and clicking buttons to retrieve meaningful detailed information and represent complicated traffic networks. Consequently, more and more designs can be sketched and created quickly, which potentially speeds up one's studies or transportation planning. Furthermore, an increased number of designs means that the urban area may have better transportation plans, thereby reducing traffic congestion and CO₂ emissions. Regarding study goals, the team plans to master in Object-Oriented Programming concepts and gain more experience with teamwork, writing documents such as javadoc and academic reports.

2 Project Overview

2.1 Description

The software **simplfi**, which stands for **Simulation Map Traffic**, is a traffic simulation tool used to create and monitor simulated representation of real-world networks. For clarity, the application's core is based on SUMO, but the difference is that it is developed to suit a wider range of target users from non-coders to experts.

2.2 Technology Stack

There are three main components the team uses for the project: **Java**, **TraaS**, and **SUMO**. Additionally, the team uses **GitHub** to create a shared repository for members to submit codes, and **Notion** to distribute tasks.

2.3 Team Roles

The below table shows the main role of each member during the milestone 1, note that although these are specific-domain roles, the team tends to overlap the work and responsibilities in order to fully experience how to work in real-world projects:

ID	Name	Main Role
1402720	Kenji Bischoff	Documentation and Analysis
1650329	Nguyen Ha Khai	Documentation and Designer
1648306	Nguyen Duy Khanh	Frontend
1651780	To Huynh Phuc	Coordinator and Technical Lead
1650910	Ninh Hoang Quan	Backend

Table 1: Team Roles

2.4 Time Plan

For the detailed time plan, readers can see our activities in this Notion page: https://www.notion.so/2a62b60367bb80a58886c6713e08ba54?v=2a62b60367bb80db8153000caff77c02&source=copy_link

3 Architecture Diagram

Here is the architecture diagram of the project:

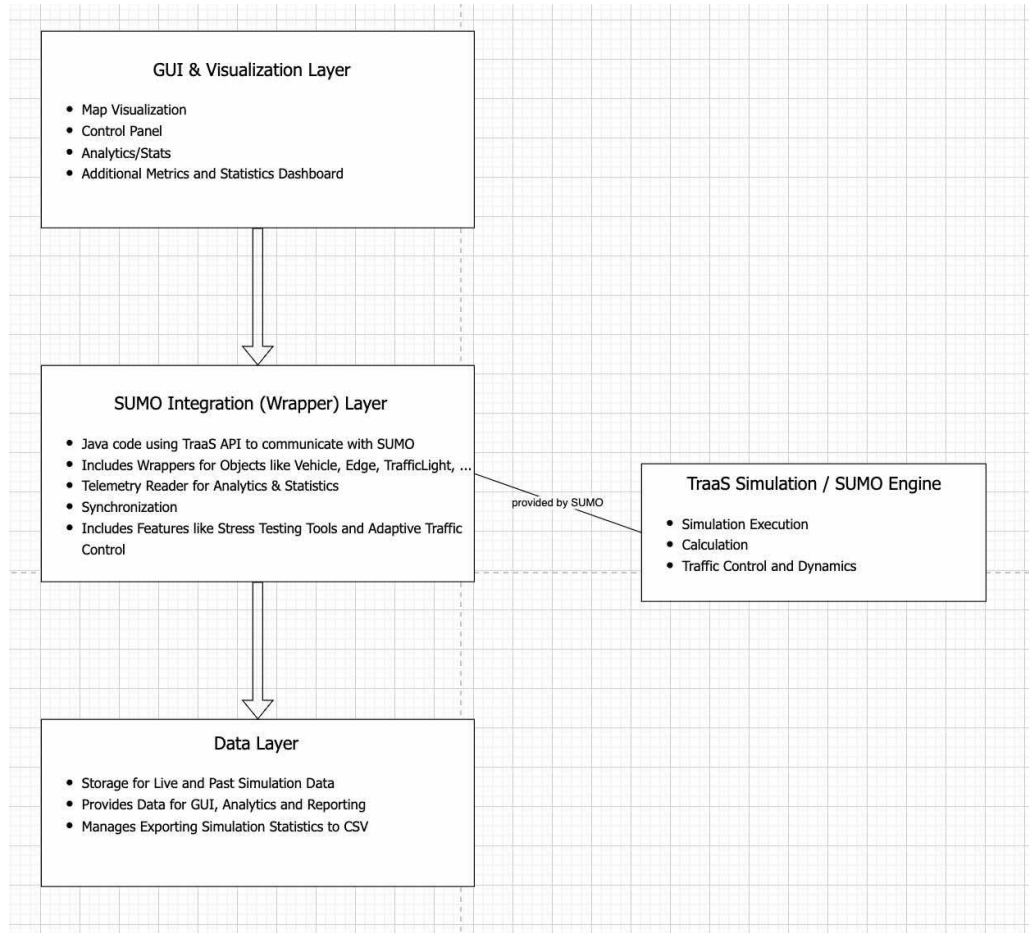


Figure 1: Architecture Diagram

From the diagram, three main layers can be observed: GUI & Visualization, SUMO Integration, and Data. Each layer provides separate methods in order to guarantee modularity and coherence in the system's design.

3.1 GUI & Visualization Layer

The main task of this layer is to present an user-friendly UI for users. In particular, it includes map visualization, multiple panels such as **Inject** so that users can add vehicles to the network. Additionally, users can move and scale the map by using keyboard shortcuts like `Ctrl =(command =)` to increase its size.

3.2 SUMO Integration (Wrapper) Layer

The layer is where TraCI connection takes place. Put differently, it is the interface where SUMO and wrapper classes communicate with each other. From there, the simulation can be executed, logical operations like numerical calculations can be performed, and network controls can be handled flawlessly.

3.3 Data Layer

As its name suggests, this layer handle data-related operations such as live storage and past storage, which is essential for exporting meaningful statistics. Also, the layer is developed to ensure the consistency and integrity of data throughout different parts of the system.

4 Class Design

4.1 Overview

Besides core wrapper classes, the team decided to draw the user interface by defining and connecting classes such as buttons, panels,... As a result, the number of classes was up to 30, which would be excessively complicated and difficult to understand if the team sketch them in one single class diagram. Therefore, a wrapper class diagram was designed to ensure the conciseness and readability.

4.2 Wrapper Class Diagram

The class diagram is shown as follows:

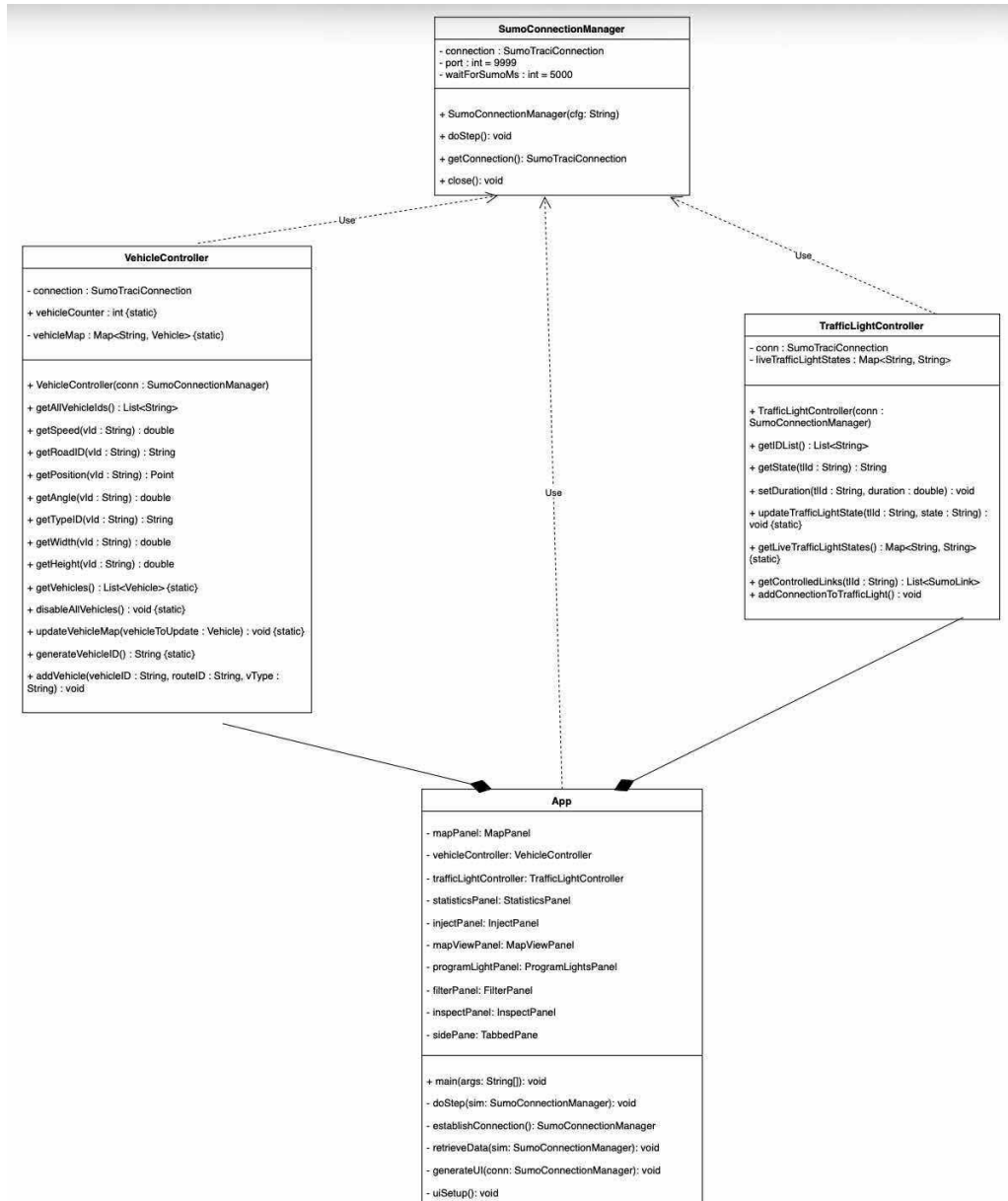


Figure 2: Wrapper Class Diagram

4.3 Class Responsibilities

- **VehicleController**: its main role is to process and retrieve detailed information regarding vehicles. To achieve this, the class has a `SumoTraciConnection` attribute `connection` used to actively connect to SUMO so that getters such as `getSpeed()`, `getRoadID()` can be implemented. Additionally, `vehicleCounter` and `vehicleMap` are developed to keep tracks of

vehicles number and mapping. Furthermore, the class provides several useful methods like `addVehicle()` for users to perform vehicle-related operations.

- **TrafficLightController**: as its name suggests, the class deals with traffic light objects, especially in the real-time scenarios since it has a live traffic light mapping: `liveTrafficLightStates`. Similar to `VehicleController`, the class establishes a connection with SUMO using the attribute `connection` and therefore is able to implement getters such as `getIDList()`, `getState()`. Besides, `updateTrafficLightState()`, `getLiveTrafficLightStates()` are provided to maintain the logic of SUMO traffic lights in the Simphi's map.

- **SumoConnectionManager**: is the class that creates and manages the TraCI connection between SUMO and the project's app. This is achieved by passing a XML configuration file to its constructor and then the program can initialize the connection by `getConnection()` method and close it safely later using `close()`. Moreover, it provides an *important* method `doStep()` which advances the simulation by one timestep.

- **App**: is where the main function resides. Particularly, it connects the frontend (`mapPanel`) and the backend (`vehicleController` and `trafficLightController`); defines the TraCI connection; and handles the connection loop so that the software can function properly.

5 GUI Mockups

Below is the intended final user interface of the project:

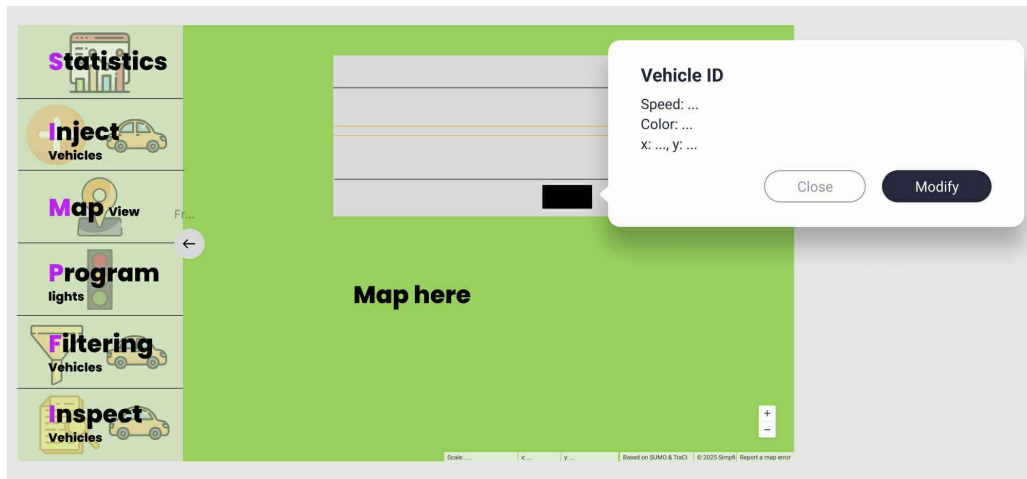


Figure 3: User Interface

6 User Guide

6.1 Introduction

6.1.1 Overview

This application provides a simple graphical interface for controlling, visualizing, and analyzing SUMO traffic simulations. Users can inject traffic, program traffic lights, and monitor simulation statistics in real time.

6.1.2 Target Audience

Our project is aimed at users working with traffic and urban mobility simulations, such as:

- Designers.
- Transportation Researchers.
- Environmental Analysts.
- Students.

6.1.3 Features

- Real Time Map Visualization.
- UI including:
 - Vehicle Injection
 - Vehicle Inspection and Filtering
 - Traffic Light Programming
 - Statistics Panel
 - Map View settings

6.2 Setup

6.2.1 Software Requirements

1. Java 17 (JDK) or higher.
2. SUMO software.
3. IDE for Java.

6.2.2 Installation Guide

Make sure that you download the versions compatible with your operating system.

1. Install Java Development Kit (JDK from oracle).
2. Install SUMO (Simulation of Urban MObility).
3. Install an IDE of your choice to run the Java project (Eclipse, VSCode, IntelliJ, ...).
4. Install `flatlaf-3.6.2.jar`.

6.2.3 Project Setup

The last step of the setup is to download our project by cloning it from GitHub. After that, open it up in your IDE.

For running the application, simply run the class "App.java".

6.3 Getting Started

After successfully setting up and running the application, you will find yourself directly on the User Interface, including various panels on the left side:

1. Statistics

2. Inject

The Inject Panel is used to create a new Vehicle and place it on Route. It displays two mandatory fields — Vehicle Type and Route — that the user must set to place a vehicle in the simulation. From the dropdown menu, select the vehicle type and its route. By pressing "Adding vehicle" you inject the vehicle on the selected route.

3. Map View

4. Program Lights

5. Filter

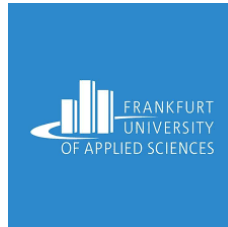
6. Inspect

On the right hand side you see the map, which you can move with the arrow keys. You can also zoom with `Ctrl =` to zoom in as well as `Ctrl -` to zoom out.

7 Conclusion

From an objective perspective, the team is progressing at a reasonable rate and therefore plausibly finish the project within the target date.

FRA-UAS
DEPARTMENT OF COMPUTER SCIENCE



REAL-TIME TRAFFIC SIMULATION WITH JAVA

SiMpFi

Instructor: Prof. Dr.-Eng. Ghadi Mahmoudi

Student 1: Kenji Bischoff - 1402720

Student 2: Nguyen Ha Khai - 1650329

Student 3: Nguyen Duy Khanh - 1648306

Student 4: To Huynh Phuc - 1651780

Student 5: Ninh Hoang Quan - 1650910



Abstract

This report provides progress, purpose, and other detailed informations about simpfi, a real-time traffic simulation app in which users can simulate real-world transportation scenarios and export useful statistics.



List of Tables

1	Team Roles	6
---	----------------------	---

List of Figures

1	Architecture Diagram	7
2	Wrapper Class Diagram	9
3	User Interface	11
4	Navigating UI	13
5	Statistics Panel	14
6	Inject Panel Interface	15
7	Map View Panel Interface	16
8	Choosing another connection in the same Intersection	17
9	Information DiaLog in Program Lights Panel	17
10	Map when entering the mouse into E45	19
11	Map when unticking E45	19
12	Inspect Panel	21
13	Statistics Panel	22
14	Inject Panel	23



Contents

1	Introduction	5
1.1	Context	5
1.2	Main Issue	5
1.3	Purpose and Goals	5
2	Project Overview	5
2.1	Description	5
2.2	Technology Stack	6
2.3	Team Roles	6
2.4	Time Plan	6
3	Architecture Diagram	7
3.1	GUI & Visualization Layer	7
3.2	SUMO Integration (Wrapper) Layer	8
3.3	Data Layer	8
4	Class Design	8
4.1	Overview	8
4.2	Wrapper Class Diagram	8
4.3	Class Responsibilities	10
5	GUI Mockups	10
6	User Guide	11
6.1	Introduction	11
6.1.1	Overview	11
6.1.2	Target Audience	11
6.1.3	Features	11
6.2	Setup	12
6.2.1	Software Requirements	12
6.2.2	Installation Guide	12
6.2.3	Project Setup	12
6.3	User Handbook	13
6.3.1	Statistics	13
6.3.2	Inject	15
6.3.3	Map View	16
6.3.4	Program Lights	16



6.3.5	Filter	18
6.3.6	Inspect	20
6.4	Technical Description	21
6.4.1	Technical Description – Statistics Panel	21
6.4.2	Technical Description – Inject Panel	22
6.4.3	Technical Description – Map View Panel	24
6.4.4	Technical Description – Program Lights Panel	24
6.4.5	Technical Description – Filter Panel	24
6.4.6	Technical Description – Inspect Panel	25
7	Conclusion	26

1 Introduction

1.1 Context

SUMO - Simulation of Urban MObility - is a traffic simulation tool used to simulate urban transportation scenarios. The application is helpful for designers, researchers who design road network and evaluate features such as CO₂ levels, traffic congestion rate, etc.

1.2 Main Issue

Although providing useful functionalities for traffic simulation, SUMO is difficult to utilize effectively. In other words, only users with programming knowledge can benefit greatly from the software.

This disadvantage can be observed by the fact that users are required to grasp a basic understanding of SUMO's configurations such as XML files and how to run command lines.

1.3 Purpose and Goals

In order to address the problem mentioned above, the team aims to create a software that leverages SUMO's features but maintains the simplicity of user interface. That is, people can perform simple operations such as drag-and-drop and clicking buttons to retrieve meaningful detailed information and represent complicated traffic networks. Consequently, more and more designs can be sketched and created quickly, which potentially speeds up one's studies or transportation planning. Furthermore, an increased number of designs means that the urban area may have better transportation plans, thereby reducing traffic congestion and CO₂ emissions. Regarding study goals, the team plans to master in Object-Oriented Programming concepts and gain more experience with teamwork, writing documents such as javadoc and academic reports.

2 Project Overview

2.1 Description

The software **simpfi**, which stands for **Simulation Map Traffic**, is a traffic simulation tool used to create and monitor simulated representation of real-world networks. For clarity, the application's core is based on SUMO, but the difference is that it is developed to suit a wider range of target users from non-coders to experts.

2.2 Technology Stack

There are three main components the team uses for the project: **Java**, **TraaS**, and **SUMO**. Additionally, the team uses **GitHub** to create a shared repository for members to submit codes, and **Notion** to distribute tasks.

2.3 Team Roles

The below table shows the main role of each member during the milestone 2, note that although these are specific-domain roles, the team tends to overlap the work and responsibilities in order to fully experience how to work in real-world projects:

ID	Name	Main Role
1402720	Kenji Bischoff	Implement Inspect Panel
1650329	Nguyen Ha Khai	Implement Filter Panel
1648306	Nguyen Duy Khanh	Implement Program-Lights Panel
1651780	To Huynh Phuc	Implement Map-View Panel
1650910	Ninh Hoang Quan	Implement Statistics Panel

Table 1: Team Roles

2.4 Time Plan

For the detailed time plan, readers can see our activities in this Notion page: https://www.notion.so/2a62b60367bb80a58886c6713e08ba54?v=2a62b60367bb80db8153000caff77c02&source=copy_link

3 Architecture Diagram

Here is the architecture diagram of the project:

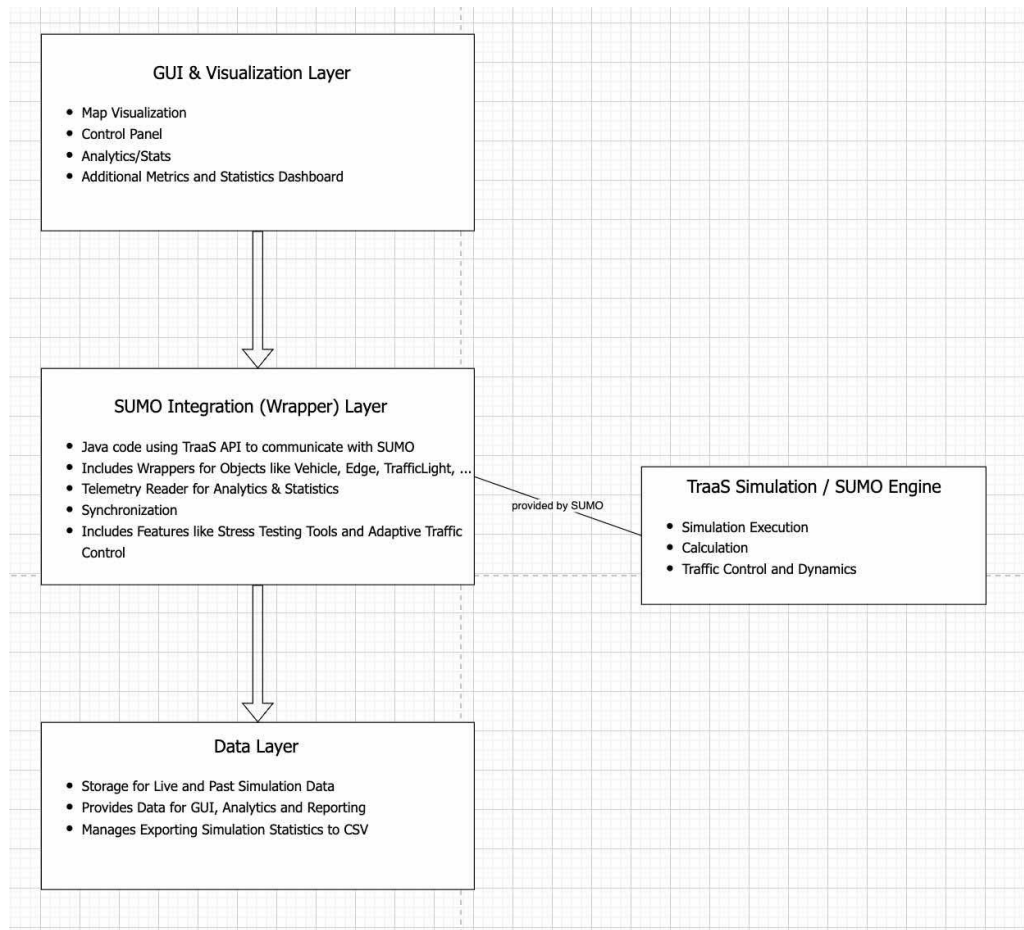


Figure 1: Architecture Diagram

From the diagram, three main layers can be observed: GUI & Visualization, SUMO Integration, and Data. Each layer provides separate methods in order to guarantee modularity and coherence in the system's design.

3.1 GUI & Visualization Layer

The main task of this layer is to present an user-friendly UI for users. In particular, it includes map visualization, multiple panels such as **Inject** so that users can add vehicles to the network. Additionally, users can move and scale the map by dragging and scrolling the mouse respectively.

3.2 SUMO Integration (Wrapper) Layer

The layer is where TraCI connection takes place. Put differently, it is the interface where SUMO and wrapper classes communicate with each other. From there, the simulation can be executed, logical operations like numerical calculations can be performed, and network controls can be handled flawlessly.

3.3 Data Layer

As its name suggests, this layer handle data-related operations such as live storage and past storage, which is essential for exporting meaningful statistics. Also, the layer is developed to ensure the consistency and integrity of data throughout different parts of the system.

4 Class Design

4.1 Overview

Besides core wrapper classes, the team decided to draw the user interface by defining and connecting classes such as buttons, panels,... As a result, the number of classes was up to 30, which would be excessively complicated and difficult to understand if the team sketch them in one single class diagram. Therefore, a wrapper class diagram was designed to ensure the conciseness and readability.

4.2 Wrapper Class Diagram

The class diagram is shown as follows:

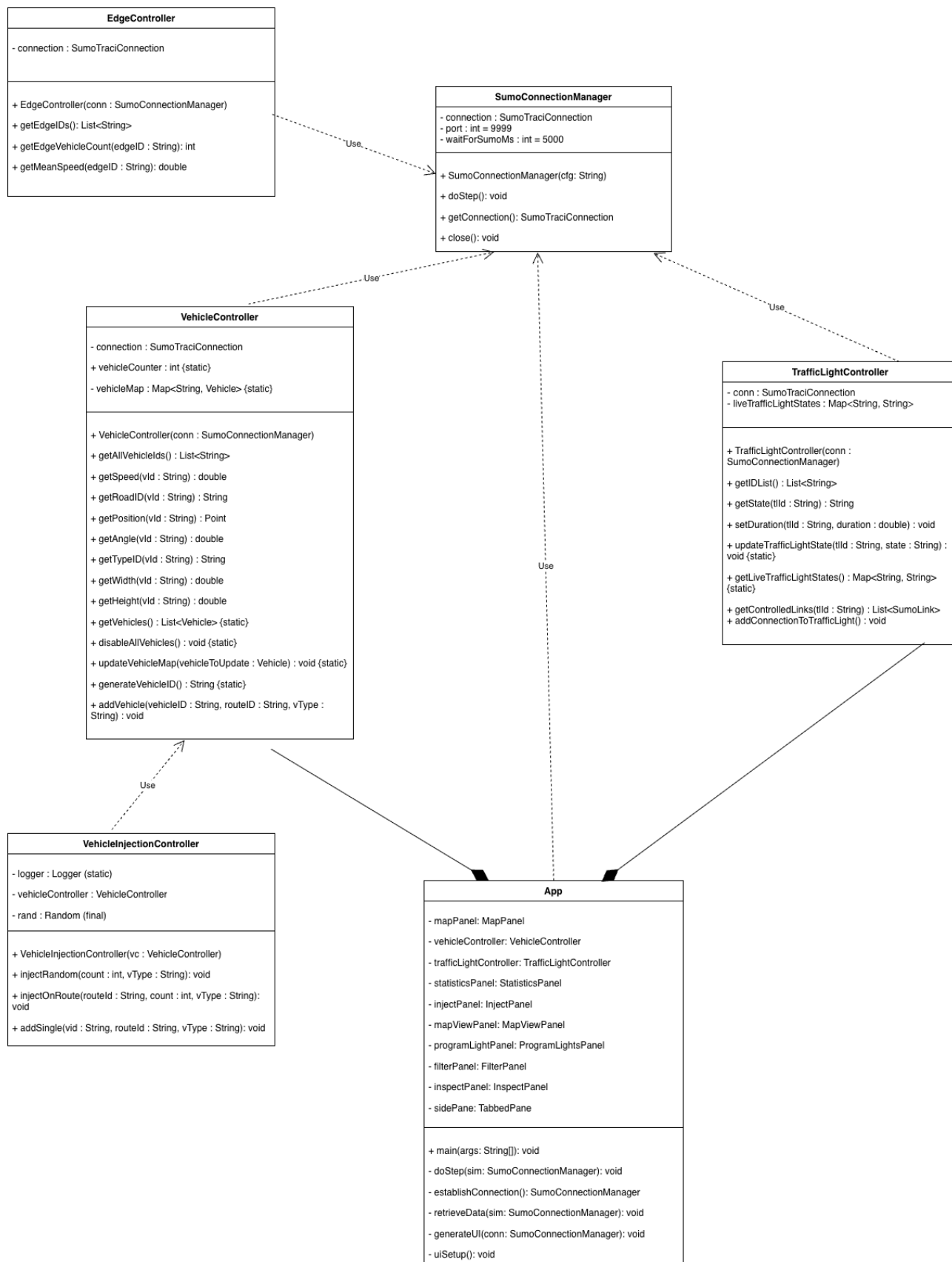


Figure 2: Wrapper Class Diagram

4.3 Class Responsibilities

- **EdgeController**: gets the information of edges and vehicles to calculate meaningful statistics such as average speed on a specific edge.
- **VehicleInjectionController**: controls different types of injections like batch, batch random or single mode.
- **VehicleController**: its main role is to process and retrieve detailed information regarding vehicles. To achieve this, the class has a `SumoTraciConnection` attribute `connection` used to actively connect to SUMO so that getters such as `getSpeed()`, `getRoadID()` can be implemented. Additionally, `vehicleCounter` and `vehicleMap` are developed to keep tracks of vehicles number and mapping. Furthermore, the class provides several useful methods like `addVehicle()` for users to perform vehicle-related operations.
- **TrafficLightController**: as its name suggests, the class deals with traffic light objects, especially in the real-time scenarios since it has a live traffic light mapping: `liveTrafficLightStates`. Similar to `VehicleController`, the class establishes a connection with SUMO using the attribute `connection` and therefore is able to implement getters such as `getIDList()`, `getState()`. Besides, `updateTrafficLightState()`, `getLiveTrafficLightStates()` are provided to maintain the logic of SUMO traffic lights in the Simpfi's map.
- **SumoConnectionManager**: is the class that creates and manages the TraCI connection between SUMO and the project's app. This is achieved by passing a XML configuration file to its constructor and then the program can initialize the connection by `getConnection()` method and close it safely later using `close()`. Moreover, it provides an *important* method `doStep()` which advances the simulation by one timestep.
- **App**: is where the main function resides. Particularly, it connects the frontend (`mapPanel`) and the backend (`vehicleController`, `trafficLightController`, ...); defines the TraCI connection; and handles the connection loop so that the software can function properly.

5 GUI Mockups

Below is the intended final user interface of the project:

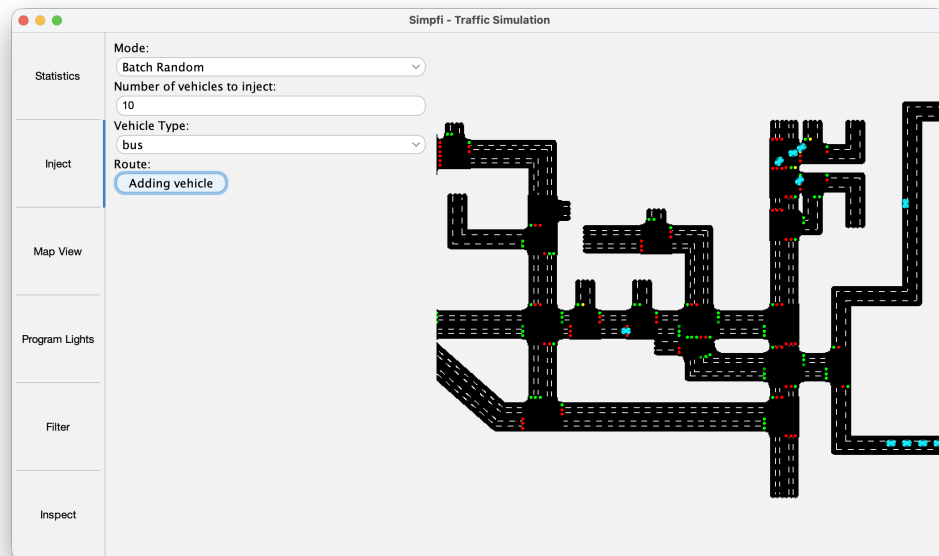


Figure 3: User Interface

6 User Guide

6.1 Introduction

6.1.1 Overview

This application provides a simple graphical interface for controlling, visualizing, and analyzing SUMO traffic simulations. Users can inject traffic, program traffic lights, and monitor simulation statistics in real time.

6.1.2 Target Audience

Our project is aimed at users working with traffic and urban mobility simulations, such as:

- Designers.
- Transportation Researchers.
- Environmental Analysts.
- Students.

6.1.3 Features

- Real Time Map Visualization.

- UI including:
 - Vehicle Injection
 - Vehicle Inspection and Filtering
 - Traffic Light Programming
 - Statistics Panel
 - Map View settings

6.2 Setup

6.2.1 Software Requirements

1. Java 17 (JDK) or higher.
2. SUMO software.
3. IDE for Java.

6.2.2 Installation Guide

Make sure that you download the versions compatible with your operating system.

1. Install Java Development Kit (JDK from oracle).
2. Install SUMO (Simulation of Urban MObility).
3. Install an IDE of your choice to run the Java project (Eclipse, VSCode, IntelliJ, ...).
4. Install `flatlaf-3.6.2.jar`, `jcommon-1.0.23`, `jfreechart-1.0.19`, TraaS.

6.2.3 Project Setup

The last step of the setup is to download our project by cloning it from GitHub. After that, open it up in your IDE.

For running the application, simply run the class "App.java".

6.3 User Handbook

The application window will be divided into 2 main sections: (1) Map Panel on the right side and (2) Control Panel on the left side.

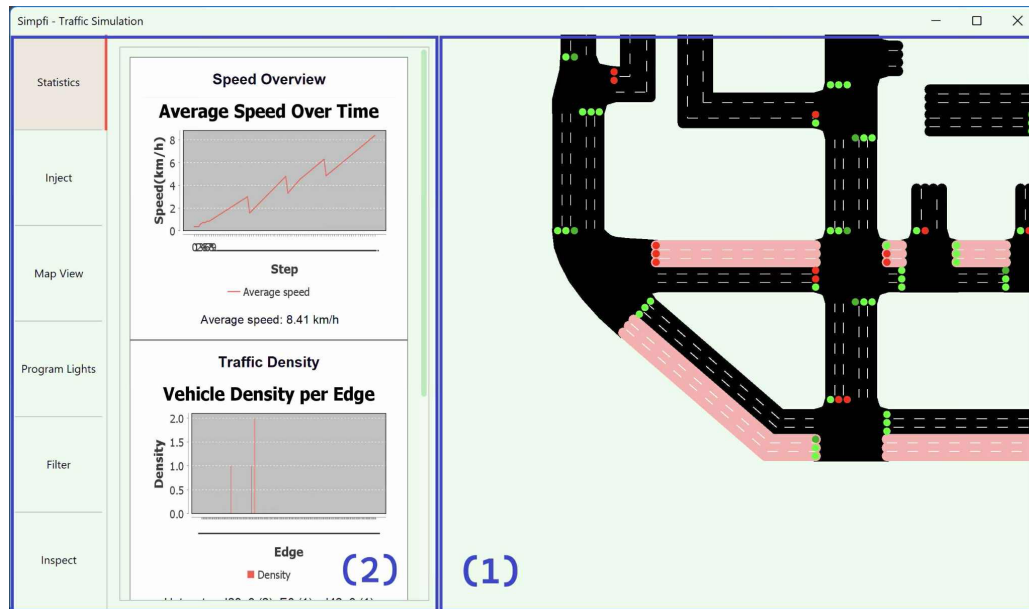


Figure 4: Navigating UI

The **Map Panel** can be manipulated in 2 ways:

- Move around with arrow keys, zoom in with “Ctrl =” and zoom out with “Ctrl -” (Not recommended since arrow keys can also be used to move around in the Control Panel and differences between US keyboard layout and German keyboard layout makes zooming more difficult).
- Move around by clicking and dragging the map area, zoom in by scrolling up and zoom out by scrolling down (Recommended).

The **Control Panel** have six sub-panels with different features:

6.3.1 Statistics

Shows a variety of charts that is updated in real time with statistics from the running simulation:

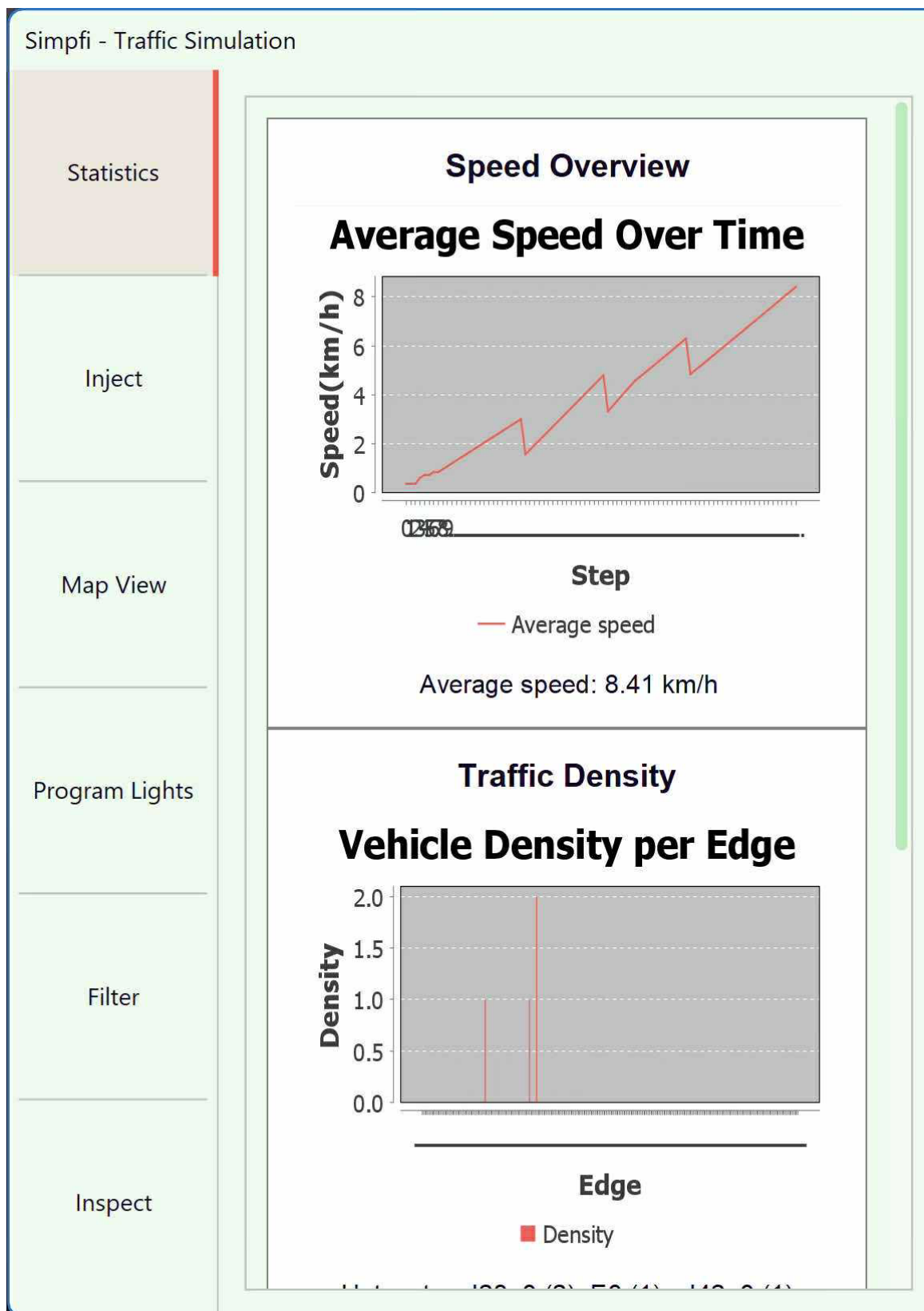


Figure 5: Statistics Panel

6.3.2 Inject

In the Inject Panel, there are four Dropdowns and one button to add vehicle to the network:

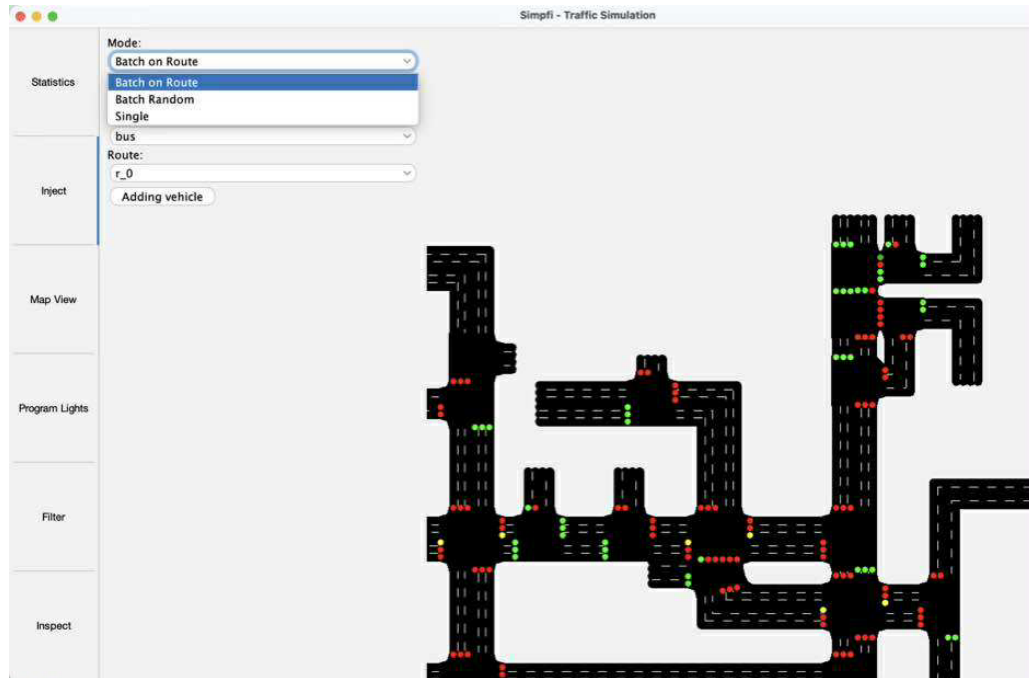


Figure 6: Inject Panel Interface

Before injection, users have to choose one of three modes: batch on route, batch random, and single. Batch On Route means that users can inject any number of vehicles they want on a specific route chosen from the Route Dropdown. For the Batch Random mode, users are allowed to add any number of vehicles to random routes. The last mode is Single mode, for which users can only inject one vehicle on one route.

After choosing the mode, users have to set a value indicating the number of vehicles that they want to inject. This Dropdown only appears when a chosen mode is Batch On Route or Batch Random. The third Dropdown is Vehicle Type, where users can choose one of five vehicle types. Each vehicle is colored differently depending on its type. Next, users have to specify which route they want their vehicles to run. This Dropdown appears only when a mode is Batch On Route or Single. Finally, users press the button "Adding vehicle" to start the process.

6.3.3 Map View

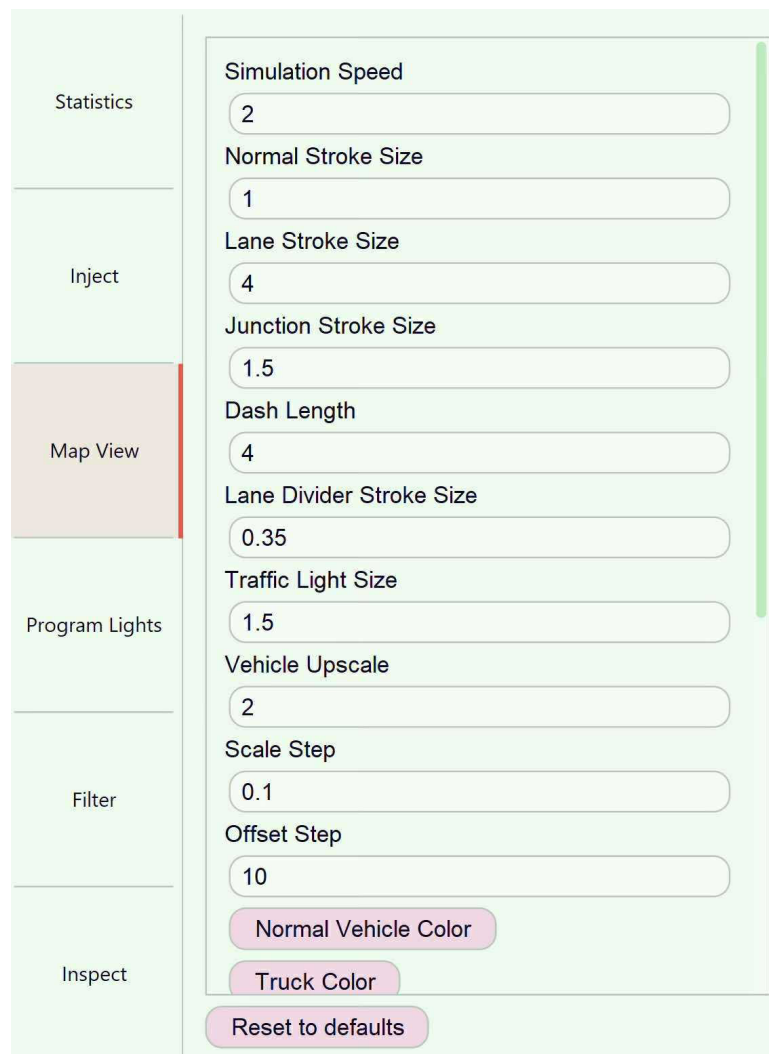


Figure 7: Map View Panel Interface

In the Map View panel, user can change many settings about the display of the map such as how big the elements are or what color is used. **Note** that set the “Simulation Speed” too high is not intended and can have unexpected consequences.

6.3.4 Program Lights

In this panel, users can interact with traffic lights in the network. To start with, users need to identify the traffic lights they want by selecting the traffic light ID in “Select intersection”.

After a intersection is chosen, one of its connections is highlighted. The intersection between two lanes connection is the junction that users choose. If users want to see other connections, simply select them in “Connection”. For example, another connection becomes highlighted after being selected:

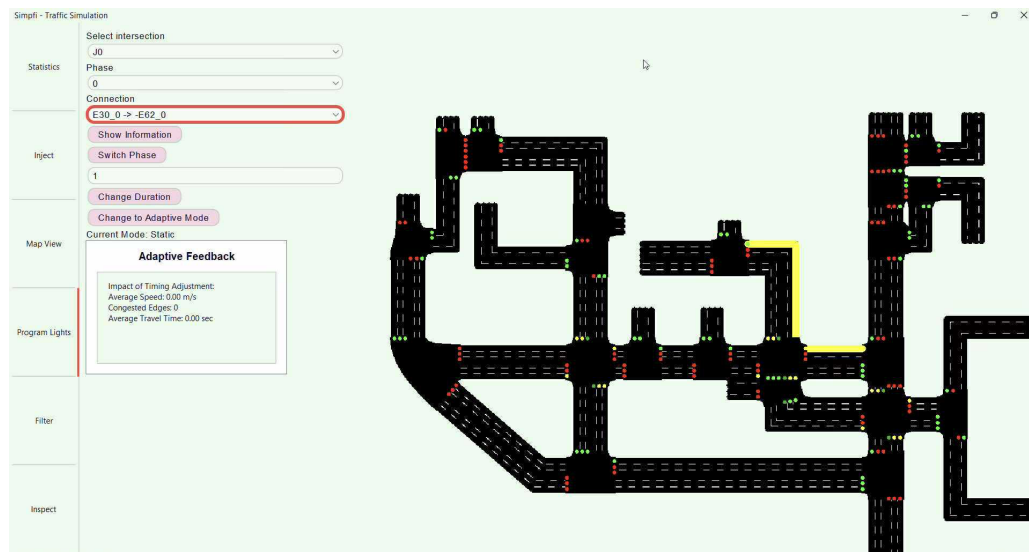


Figure 8: Choosing another connection in the same Intersection

If users want to view all the information including the state of the traffic light, the current phase, the signal of each connection, and the remaining time of the current duration, Users can click on “Show information”.

After that, a pop-up dialog will appear on the screen and update by real time in traffic light. And the signal of traffic light will represent the status light of that connection. Here “r” is red, “y” is yellow, and “g” is green. Sometimes, users will see big “G”, which represents for green light, but higher priority than “g”.

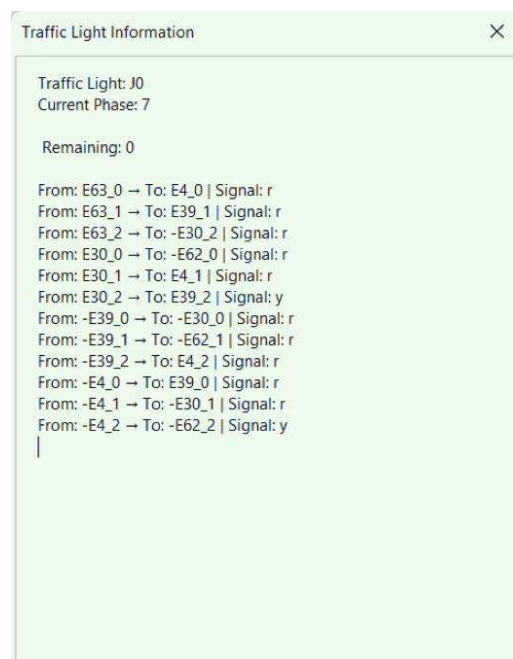


Figure 9: Information DiaLog in Program Lights Panel

If users want to switch the desired phases immediately. First, users need to choose the traffic light ID in “Select intersection” and then choose the phase number in “Phase” and click “Switch Phase” after that to trigger the event. Similarly, if users want to set new duration for specific phases of the traffic light. First, select traffic light ID in “Select intersection”. And then select the phase user want to choose in “Phase”. Then enter the time value which is the new duration for the phase user choose. And finally, click “Change Duration”. Additionally, this panel introduce two different traffic light modes: static and adaptive mode. Users can freely switch between two modes by clicking “Change to Adaptive Mode” or “Change to Static Mode”. Here “Static Mode” is when the logic or program of the traffic light is predefined from the *XML* file. So, there is no change in the definition of the program (e.g. phases, states, signals, duration, etc.). If users tend to choose the mode to “Adaptive” mode. The traffic light may change based on the congestion of the traffic. Here, the team added the logic that if any traffic light has the lane which the number of vehicles of that lane is larger than 2, then the duration of current phase of that traffic light is increased to 5 (In other words, take the default duration and plus 5). This helps users to track the traffic and have a comparison between static mode and adaptive mode.

6.3.5 Filter

This panel provides three main features:

- **vehicle-type filter:** users can see a list of checkboxes showing different types of vehicles. Vehicles whose type is not ticked will not be painted on the map.
- **road filter:** with this filter, users are able to filter out roads, which are the combinations of edges and their sub-edges(not include junction edges). Designed similarly like the **vehicle-type filter**, users check the boxes to opt in roads where vehicles are visible. The only difference is that users can enter the mouse into each checkbox to see the corresponding road. For example, E45 is entered:

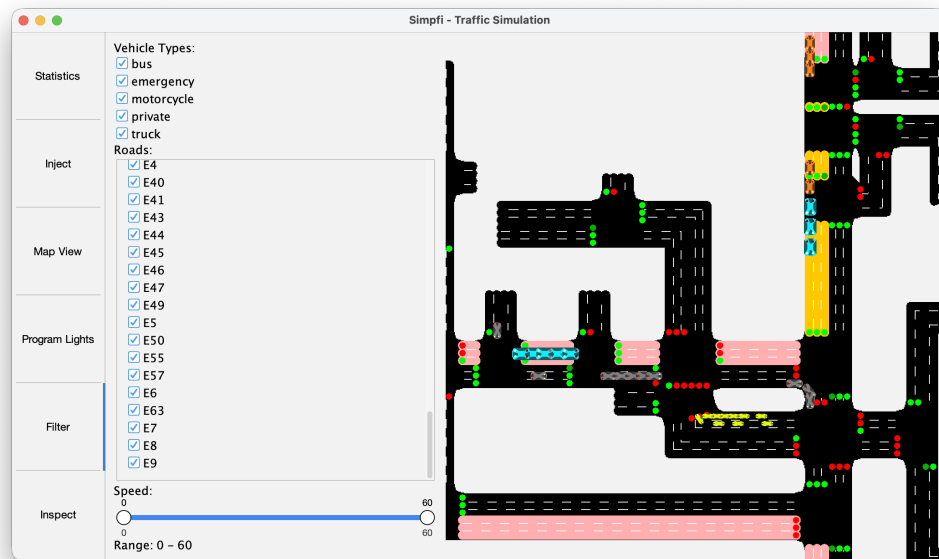


Figure 10: Map when entering the mouse into E45

Then when E45 is unticked, vehicles become invisible in that road:

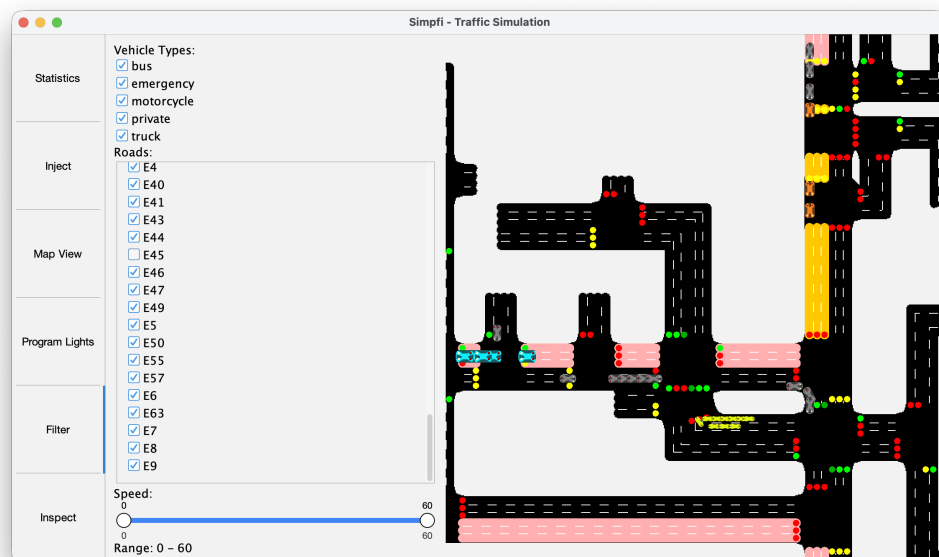


Figure 11: Map when unticking E45

- **speed filter:** Different from the two previous filters, this filter use a range slider to filter out vehicles within a specific range. The defined range is set from 0 to 60(m/s), which is derived from German speed limits. Users can drag two thumbs to set the desired speed range for visible vehicles.

6.3.6 Inspect

With the **Inspect Panel**, users can select vehicles directly from the Simulation by clicking on it.

The user can switch between two modes by pressing the **Change Mode** Button on the bottom: The **Pan mode** is used to drag the map around. After changing to the **Select mode**, the user will be able to select a vehicle on the map by clicking on it.

The list on the top left of the panel shows the currently selected Vehicles. To the right there is a **Select All button** to select all vehicles, which are currently in the Simulation. Use the **Clear button** below to clear the List.

If the user chooses a selected vehicle from the upper box, this element will be highlighted and the corresponding stats will show in the lower Box.

The list can also be sorted by using the Group By dropdown. The user can group the selected vehicles by **Vehicle Type**, **Color**, **Speed** or **Route**. To see the list without grouping, select **None**.

The stat **Vehicle Type** shows one of the following vehicles: Bus (blue), Emergency (orange), Motorcycle (pink), Private (yellow), Truck (grey).

The stat **Color** shows the color in an RGB format, representing the Color components Red, Green, Blue from 0 to 255. Example: R:255, G:0, B:0 → maximum (255) amount of Red, 0 Green and Blue

The **Speed** is the only dynamic statistic in the list. It gets updated every 500ms. The speed is given in km/h.

Max Speed shows the maximum speed the vehicle reaches in the simulation.

Acceleration is given in m/s^2 .

The **Distance Traveled** stat shows the covered distance in this simulation and is given in meters.

The **Route** stat shows the connected edges forming the vehicles route.

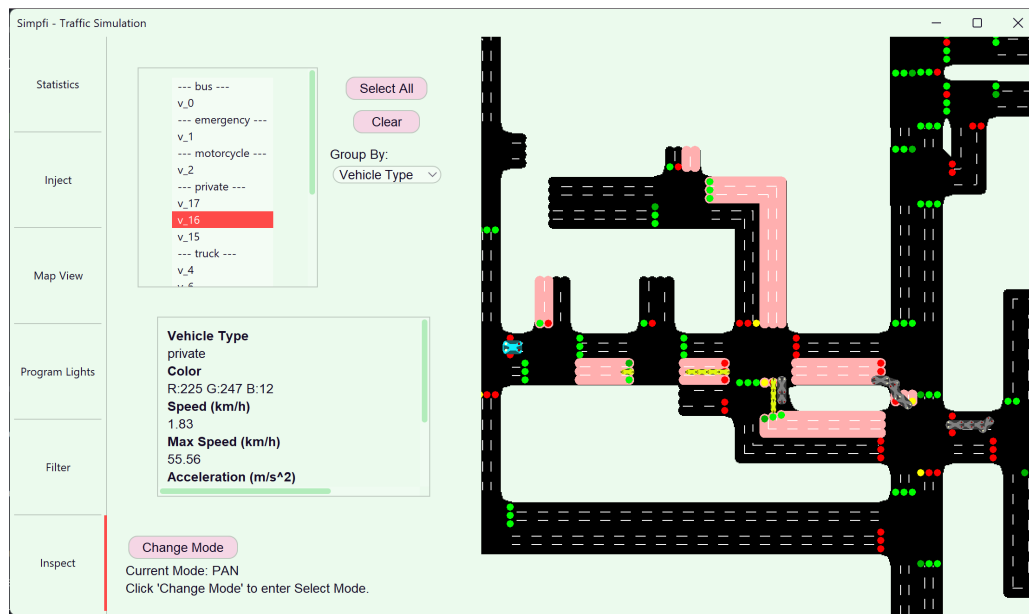


Figure 12: Inspect Panel

As you can see in the picture, the list is currently grouped by Vehicle Type. Each batch of vehicles with the same type are headed with their types name.

6.4 Technical Description

Below are the six panels with a more technical explanation:

6.4.1 Technical Description – Statistics Panel

In the Statistics feature, the average speed, vehicle density per edge, and travel time distribution are visualized by 3 live charts: line chart, bar chart, and histogram. First of all, the average speed is computed by summing all vehicles' velocities and dividing by the number of vehicles running on the network via the `getAverageSpeed()` method. The second data is the traffic density, which simulates the density of vehicles on each edge using a bar chart and results in congested hotspots – 3 edges with the largest number of vehicles. Particularly, the number of vehicles is collected on each edge per step by traversing all edge IDs and storing them in a hash map, which is the `edgeVehicleCount`. Then, this data is put into the `vehicleDataset` for plotting a live bar chart. Finally, the travel time data is computed by subtraction of the current step from the start step. Then, it is stored in a list `travelTimes` for the travel time distribution visualization.

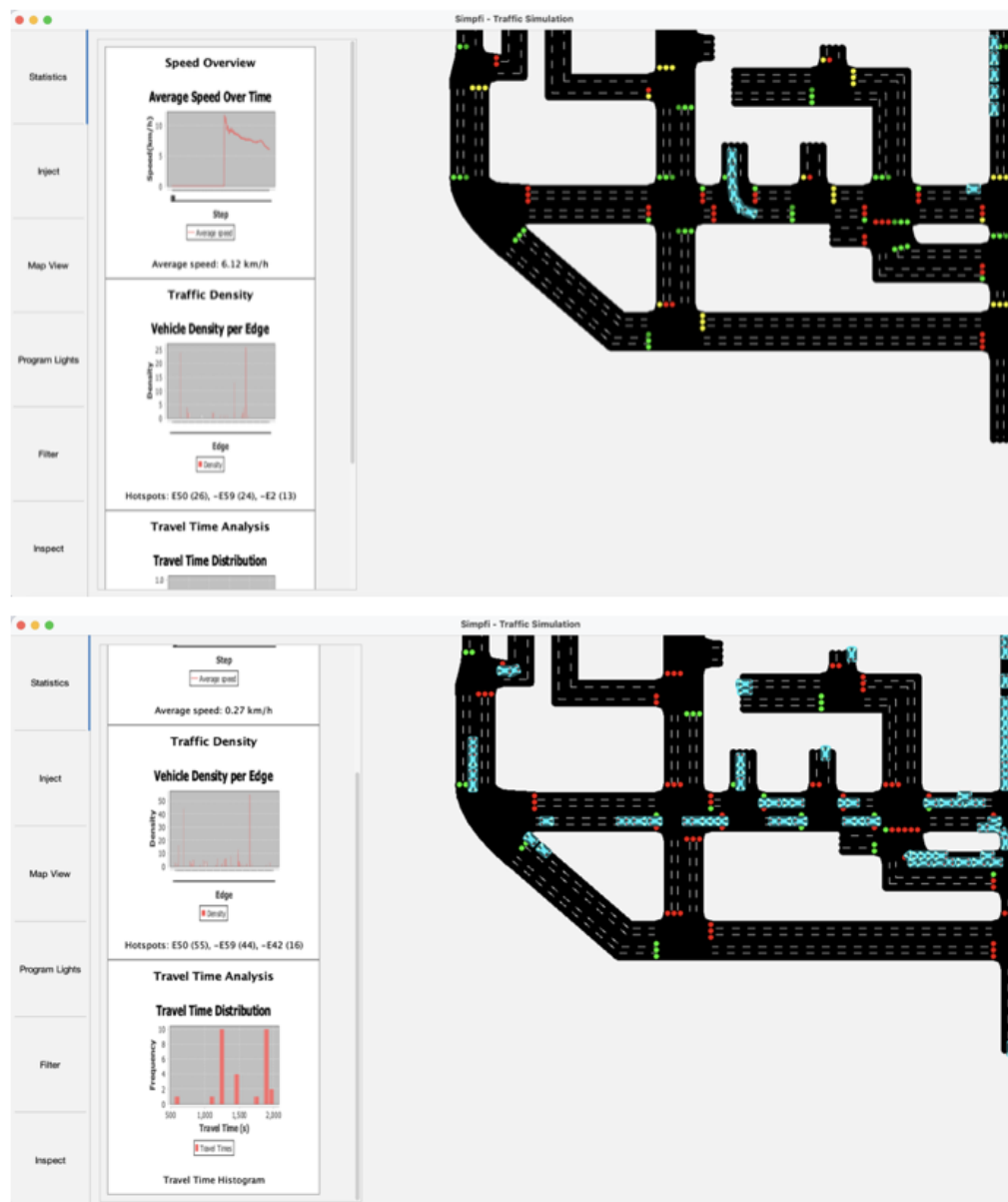


Figure 13: Statistics Panel

6.4.2 Technical Description – Inject Panel

The InjectPanel class is a Java Swing-based user interface for injecting vehicles. It inherits the Panel class and allows for selection of vehicle types, routes, injection mode, and the number of vehicles to inject. The InjectPanel allows users to dynamically inject vehicles into a network during simulation time. It supports three modes of injection:

- **Batch On Route** – inject multiple vehicles across a specific route
- **Batch Random** – inject multiple vehicles across a network
- **Single** – inject a selected vehicle across a chosen route

The image below shows 5 key components of the Inject Panel feature:

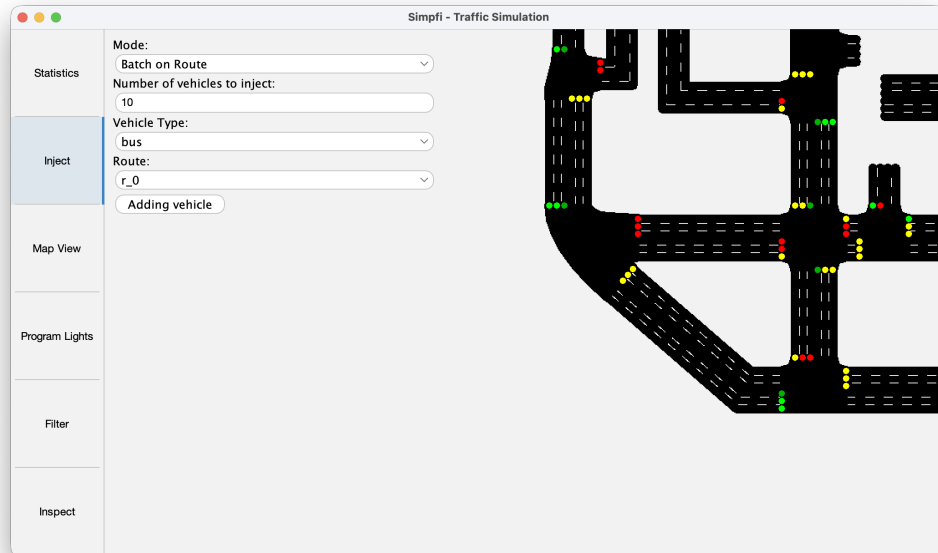


Figure 14: Inject Panel

1. **Mode Dropdown (modeDropdown):** Provides 3 options – Batch On Route, Batch Random, and Single. It determines whether users want to inject multiple vehicles into a selected or a random route, or even a single vehicle into a specific route when clicking on the “Add vehicle” button.
2. **Count Field (countField):** Provides a text area allowing users to type any number of vehicles they want to add to a network in batch modes. In a single mode, it is hidden dynamically.
3. **Vehicle Type Dropdown (vehicleTypeDropdown):** allows users to select a vehicle type from a list of vehicle types obtained from `Settings.network.getVehicleTypes()`, which is wrapped by the `getAllVehicleTypesAsString()` method.
4. **Route Dropdown (routeDropdown):** allows users to select a route for vehicle injection from a list of route IDs achieved from `Settings.network.getRoutes()`, which is wrapped by the `getAllRouteIdsAsStrings()` method.
5. **Add Vehicle button (addVehicleBtn):** Executes the vehicle injection process via mouse click in a separate Thread to prevent blocking the EDT from Java Swing.

The `addVehicle()` method handles vehicle injection and reads users’ input for vehicle mode, vehicle count, vehicle type, and route. Then the method executes a thread for injection and

divides three modes into three cases using three different methods obtained from the `VehicleInjectionController` class.

6.4.3 Technical Description – Map View Panel

In the map view panel, many attributes of the map can be changed freely by the user. This is possible by updating an internal variable that can be accessed by all classes and functions so that the map drawing can be consistent. In order to implement the "Reset defaults" button, another list of constants are stored with the default values, and when the button is pressed, all properties of the map are set to the original default value.

6.4.4 Technical Description – Program Lights Panel

The Program Light Panel provides access to all traffic light information, including the junction ID, all lane connections, and the traffic light programs defined in SUMO. These attributes are retrieved through SUMO's junction calls. Since the drawing interacts with the SUMO API (TrasS), traffic light states can be changed dynamically by calling the SUMO traffic light interface. User interface elements such as drop-down menus, buttons, text areas, and dialogs are implemented by extending Java Swing classes. The remaining duration of the current phase of specific traffic light is computed by retrieving the next switch time from SUMO and subtracting it by the current time. The signal of each traffic light connection is parsed from the traffic light state String by splitting it into individual characters, where each character represents the signal assigned to a specific connection. Moreover, to ensure consistency and accuracy, both the remaining duration and the signal of the traffic light are updated in real-time and displayed in "Show Information". Finally, in order to implement the logic of "Adaptive mode", the map is continuously updated in real time. In this mode, the update logic adjusts the traffic light phase duration based on the current traffic flow conditions, enabling the system to adapt automatically and maintain flexible traffic signal control.

6.4.5 Technical Description – Filter Panel

For the implementation of the first two filters(vehicle-type and road), the team defined a `filterFlag` in `VehicleType.java` and `Road.java`. After creating the dropdowns, which extend `JComboBox`, listeners were added to those components to update the `filterFlag` when there are updates. As a result, the map will only draw vehicles whose `filterFlag` is set to `true`.

In addition to two filters above, speed filter is implemented in a more simple approach, since each vehicle already has its own `getSpeed()` method and the code just needs to check if its speed falls within the limited range before drawing. However, the difficult part is the Range Slider,

which is not supported in Java Swing. Therefore, the team developed a new `RangeSlider` object that inherits from `JComponent` to display it and let users freely choose the desired speed range. Furthermore, the maximum speed was set to 60 m/s manually because the team could not find any specific documentation about vehicles' maximum velocity in SUMO, so this figure was decided based on our research about speed limits in Germany.

6.4.6 Technical Description – Inspect Panel

The Inspect Panel allows users to inspect the vehicles in the simulation. Users can toggle between two modes:

- **Pan Mode:** Enables map navigation by dragging.
- **Select Mode:** Allows selecting a vehicle on the map with a click.

The **Select Mode** feature is realized by a Mouse Listener to get the coordinates of the selected part of the map. These coordinates are then translated to world coordinates to be utilized to find the nearest vehicle in a range of 20 units.

The top-left list displays the currently selected vehicles. Users can use the **Select All** button to select all vehicles in the simulation or the **Clear** button to clear the list. Selecting a vehicle from the list marks it and shows its statistics in the lower box. The list can be sorted using the Group By dropdown. Users can group the selected vehicles by Vehicle Type, Color, Speed, or Route. To display the list without grouping, select None. The speed headers, which appear when grouping by route are not updated live, because this would be too demanding.

Displayed vehicle statistics include:

- **Vehicle Type:** Bus, Emergency, Motorcycle, Private, Truck.
- **Color:** RGB format (0-255) representing Red, Green, and Blue components.
- **Speed:** Updated every 500 ms, in km/h.
- **Max Speed:** Maximum speed reached during the simulation.
- **Acceleration:** In m/s^2 .
- **Distance Traveled:** Covered distance in meters.
- **Route:** Sequence of connected edges forming the vehicle's route.

7 Conclusion

From an objective perspective, the team is progressing at a high rate and therefore plausibly finish the project before the target date.

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Huynh Phuc To

First and last name



Frankfurt, 18.01.2026

City, date and signature

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Duy Khanh Nguyen

First and last name



Frankfurt, 18.01.2026

City, date and signature

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Kenji Bischoff

First and last name



Frankfurt, 18.01.2026

City, date and signature

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Hoang Quan Ninh

First and last name



Frankfurt, 18.01.2026

City, date and signature

I hereby declare that the submitted project is my own unaided work or the unaided work of our team. All direct or indirect sources used are acknowledged as references.

I am aware that the project in digital form can be examined for the use of unauthorized aid and in order to determine whether the project as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future projects submitted. Further rights of reproduction and usage, however, are not granted here.

This work was not previously presented to another examination board and has not been published.

Ha Khai Nguyen

First and last name



Frankfurt, 18.01.2026

City, date and signature